



Escuela Técnica Superior
de Ingeniería Informática

Grado en Ingeniería del Software

Curso 2025-2026

Trabajo Fin de Grado

**WRAPITUP PLANNER: PLATAFORMA PARA LA
ORGANIZACIÓN ACADÉMICA Y LA
COLABORACIÓN ENTRE ESTUDIANTES**

Autor: Arturo Enrique Gutiérrez Mirandona

Tutor: Micael Gallego Carrillo



©2026 Arturo Enrique Gutiérrez Mirandona

Algunos derechos reservados

Este documento se distribuye bajo la licencia
“Atribución-CompartirIgual 4.0 Internacional” de Creative Commons,
disponible en

<https://creativecommons.org/licenses/by-sa/4.0/deed.es>

Agradecimientos

Quiero expresar mi profundo agradecimiento a todas las personas que me han acompañado a lo largo de esta etapa. A mi madre, por su constante apoyo, su paciencia en mis momentos difíciles y la confianza que siempre ha puesto en mí. A mis compañeros, por los momentos compartidos estos últimos años y por estar a mi lado en las etapas mas estresantes. A mis profesores, por su dedicación, su guía y por transmitirme todos sus conocimientos. Gracias a cada uno de ellos, este camino ha sido posible y he podido llegar hasta aquí.

Resumen

Este documento describe el proceso de desarrollo de WrapItUp Planner, una aplicación web orientada a la organización del estudio y la gestión de contenidos académicos, cubriendo de forma estructurada todas las fases del proyecto, desde la definición de los objetivos hasta la implementación final.

A lo largo de esta memoria se recogen las principales etapas del ciclo de vida del software, incluyendo el análisis, diseño, desarrollo, pruebas y despliegue del sistema. También, se presentan las decisiones técnicas adoptadas y las herramientas utilizadas durante el proceso.

El objetivo de esta memoria es documentar de manera completa el trabajo realizado, así como las metodologías y prácticas de ingeniería del software aplicadas a lo largo del desarrollo de una plataforma orientada a mejorar la organización del estudio.

El documento se estructura en varios capítulos que recogen de forma secuencial el desarrollo del proyecto. Primero, se presenta la introducción y los objetivos, donde se contextualiza el problema y se definen las metas funcionales y técnicas del sistema. A continuación, se expone la metodología seguida durante el desarrollo. Posteriormente, se describe el análisis funcional de la aplicación, detallando sus principales módulos y funcionalidades. El capítulo de descripción informática aborda los aspectos técnicos del sistema, incluyendo tecnologías, arquitectura, implementación, control de calidad y despliegue. Finalmente, se presentan las conclusiones del proyecto y las posibles mejoras que podría tener el proyecto en caso de seguir con el desarrollo.

Palabras clave:

- Aplicación web
- Single Page Application (SPA)
- Spring Boot
- API REST
- Organización academica
- Inteligencia artificial

Índice de contenidos

Índice de tablas

Índice de figuras

Índice de códigos	1
1 Introducción y objetivos	3
1.1 Contexto	3
1.2 Estado del arte	4
1.3 Objetivos	5
1.3.1 Objetivos funcionales	5
1.3.2 Objetivos técnicos	7
2 Metodología	9
3 Descripción funcional	12
3.1 Descripción funcional	13
3.1.1 Funcionalidades básicas	13
3.1.2 Funcionalidades intermedias	14
3.1.3 Funcionalidades avanzadas	16
3.1.4 Recursos adicionales	17
4 Descripción Informática	20
4.1 Contexto	22
4.2 Tecnologías	23
4.2.1 Angular	23
4.2.2 Spring Boot	23
4.2.3 Java 21	24
4.2.4 Maven	24
4.2.5 ngx-charts	24
4.2.6 SQL y MySQL	24
4.2.7 Frameworks de pruebas	24
4.2.8 OpenAI GPT-4o	25
4.3 Herramientas	25

4.3.1	Visual Studio Code (VSCode)	25
4.3.2	Postman	25
4.3.3	GitHub	26
4.3.4	GitHub Projects	26
4.3.5	SonarQube / SonarCloud	26
4.3.6	JaCoCo	26
4.3.7	OpenAPI	26
4.4	Arquitectura	27
4.4.1	Despliegue	27
4.4.2	Modelo del dominio	27
4.4.3	API REST	28
4.4.4	Arquitectura del servidor	33
4.4.5	Arquitectura del cliente	34
4.5	Implementación	36
4.5.1	Vistas de calendario: <i>Month View</i> y <i>Day View</i>	36
4.5.2	Mapa de calor de tareas	37
4.5.3	Integración de inteligencia artificial	39
4.5.4	Procesamiento de documentos y extracción de texto	40
4.6	Control de calidad	41
4.6.1	Tipos de pruebas	42
4.6.2	Cobertura de pruebas	43
4.7	Despliegue	43
4.7.1	Empaquetado y distribución	43
4.7.2	Distribución	44
4.7.3	Despliegue remoto (URJC)	44
4.8	Proceso de desarrollo	45
4.8.1	Gestión de tareas	45
4.8.2	Git	46
4.8.3	Integración y entrega continua	47
4.8.4	Versionado	48
5	Conclusiones y trabajos futuros	51
5.1	Conclusiones	51
5.1.1	Cumplimiento de objetivos	51
5.1.2	Valoración personal del proyecto	52
5.2	Trabajos futuros	52
	Bibliografía	55
	Anexos	57
A	Funcionalidades detalladas	59

A.1	Gestión académica y planificación personal	59
A.1.1	Calendario interactivo	59
A.1.2	Sistema de tareas diarias	60
A.1.3	Mapa de calor de actividad	61
A.2	Gestión de notas y contenidos	62
A.2.1	Gestión de notas	62
A.2.2	Subida y procesamiento de documentos	64
A.3	Aprendizaje y seguimiento del progreso	64
A.3.1	Cuestionarios interactivos	65
A.3.2	Seguimiento del progreso	65
A.4	Interacción social y colaboración	66
A.4.1	Sistema de comentarios	66
A.4.2	Sistema de reportes	67
A.5	Administración y moderación	67
A.5.1	Panel de administración	67
A.5.2	Gestión de usuarios	68
B	Instrucciones para ejecutar la aplicación	70
B.1	Requisitos previos	70
B.2	Ejecución de la aplicación	71
B.3	Configuración del entorno	71
B.4	Acceso a la aplicación	72
B.5	Credenciales de ejemplo	72
B.6	Datos de ejemplo	72
C	Ejecución y edición de código	74
C.1	Clonado del repositorio	74
C.2	Ejecución del sistema	74
C.2.1	Base de datos y servicios	74
C.2.2	Ejecución del backend (Spring Boot)	75
C.2.3	Ejecución del frontend (Angular)	75
C.2.4	Herramientas de desarrollo	75
C.2.5	Ejecución de pruebas	76

Índice de tablas

2.1	Planificación temporal del proyecto	10
4.1	Resumen general de las características del sistema Wrap-It-Up Planner	23
B.1	Credenciales de acceso de los usuarios de prueba	72

Índice de figuras

2.1	Diagrama de Gantt	10
3.1	Perfil de usuario	13
3.2	Dashboard de notas	14
3.3	Vista mensual del calendario interactivo	14
3.4	Vista diaria del calendario y tareas tipo to-do list	15
3.5	Panel de administración con gestión de reportes	16
3.6	Pantalla de creación y subida de documentos	16
3.7	Cuestionario tipo test generado mediante IA	17
3.8	Gráfico de evolución del rendimiento del usuario	17
4.1	Modelo de dominio del sistema	28
4.2	Arquitectura del servidor	34
4.3	Arquitectura del cliente	35
4.4	Cobertura y análisis estático del backend en SonarCloud	43
4.5	Cobertura de pruebas del frontend generada con Karma	43
4.6	Tablero de gestión de tareas en GitHub Projects	46
4.7	Evolución de commits del repositorio	47
A.1	Vista mensual del calendario interactivo	60
A.2	Vista diaria del calendario con sistema de tareas	61
A.3	Mapa de calor integrado en el perfil de usuario	62
A.4	Dashboard de gestión de notas	63
A.5	Funcionalidad de compartición de notas	63
A.6	Pantalla de subida y procesamiento de documentos	64
A.7	Cuestionario interactivo generado mediante inteligencia artificial	65
A.8	Gráfico de evolución del rendimiento en cuestionarios	66
A.9	Sistema de comentarios en una nota	67
A.10	Panel de administración del sistema	68
A.11	Pantalla de usuario bloqueado	69

Índice de códigos

4.1	Generacion de la cuadrícula mensual	36
4.2	Apertura de la vista diaria	37
4.3	Generación de semanas en el mapa de calor	38
4.4	Cálculo del color del mapa de calor	39
4.5	Construcción del prompt en OpenAIService	40
4.6	Extracción de texto desde archivos PDF usando PDFBox	41
B.1	Comandos de despliegue de la aplicación	71
B.2	Variables de entorno principales	71
C.1	Clonado del repositorio	74
C.2	Ejecución del backend	75
C.3	Ejecución del frontend	75
C.4	Pruebas del frontend	76
C.5	Ejecución de pruebas del backend	77

1

Introducción y objetivos

Este capítulo presenta el contexto que motivó el desarrollo de WrapItUp Planner, así como un análisis de las herramientas existentes relacionadas con el estudio y la organización académica. Además, se describen los objetivos funcionales y técnicos que han guiado el diseño e implementación de la aplicación.

1.1 Contexto

La creación del proyecto WrapItUp Planner nace de una necesidad real y cotidiana dentro de contextos académicos, particularmente, la dificultad que sufren muchos estudiantes al organizar su tiempo, gestionar sus tareas y mantenerse al día con todo el material de estudio que constantemente se acumula. Este problema, vivido también por mi persona, motivó la creación de una plataforma que cubriera las necesidades no solo del propio autor, sino de todos quienes buscan una forma más clara, accesible y centralizada de gestionar su vida académica.

El público objetivo de la aplicación cubre a estudiantes de cualquier nivel educativo, sin restricciones de edad, disciplina o experiencia con la tecnología. La meta al desarrollar WrapItUp Planner siempre fue ofrecer una herramienta intuitiva y útil para cualquier persona que desee mejorar su organización y reforzar su conocimiento.

La plataforma se divide dos áreas principales que cubren las necesidades esenciales de organización y estudio del alumnado. Por un lado, la sección de apuntes

permite a los usuarios crear, almacenar y compartir resúmenes de su material educativo, ya sea elaborándolos manualmente o mediante herramientas de inteligencia artificial que facilitan la síntesis y comprensión de los contenidos. Además, incorpora la generación de preguntas tipo test en formato *quiz* para reforzar el aprendizaje de forma interactiva. Por otro lado, el calendario integrado ofrece un espacio donde organizar el día a día académico mediante listas de tareas y eventos, dando una visión clara y estructurada de las obligaciones del estudiante. La existencia de estas secciones forman un entorno unificado que favorece la organización, el seguimiento del estudio y el fortalecimiento del conocimiento.

La relevancia del desarrollo de este proyecto radica en que combina en un único lugar organización, estudio y colaboración, los tres pilares fundamentales para el rendimiento académico. Al centralizar estas funciones, la plataforma busca reducir la necesidad de acceder a herramientas externas para la más óptima organización, mejorar la eficiencia del estudiante y fomentar hábitos de estudio que finalmente mejoren sus capacidades. WrapItUp Planner se plantea así como un apoyo unificado para quienes desean mantener su material al día, reforzar los contenidos vistos en clase y gestionar su carga académica.

1.2 Estado del arte

En la actualidad, existen múltiples herramientas digitales orientadas al estudio y la productividad, muchas de ellas ofrecen funcionalidades relacionadas con la organización, como la toma de apuntes o el uso de inteligencia artificial con fines educativos. No obstante, cada una de estas herramientas presenta enfoques diferentes que nos ofrecen el contexto sobre el cual nace la necesidad de integrar estas funcionalidades en un entorno único y cómodo para el usuario.

En primer lugar, Google NotebookLM [1] es una herramienta desarrollada por Google conocida por su fuerte integración con la inteligencia artificial. Su funcionamiento se basa en permitir al usuario cargar información y realizar preguntas directamente sobre ese contenido, generando respuestas contextualizadas con el material proporcionado. Además, ofrece utilidades avanzadas como la creación de audios explicativos, vídeos, mapas mentales, *quizzes* y otros recursos generados mediante inteligencia artificial. A pesar de su versatilidad y potencia, NotebookLM está orientada, principalmente, a la interacción con IA y no a la elaboración manual de resúmenes personalizados, lo que limita la capacidad del usuario para construir sus propios apuntes de forma estructurada y personalizada. Además, aunque Google Calendar existe como herramienta independiente, no se encuentra integrado directamente dentro de NotebookLM, lo que fragmenta la experiencia de organización académica.

Por otro lado, Todoist [2] ha sido una de las principales inspiraciones del proyecto. Esta es una aplicación centrada en la gestión de tareas, recordatorios y

planificación diaria, con la posibilidad de sincronizarse con calendarios externos como Google Calendar u Microsoft Outlook. Su enfoque está claramente orientado a la productividad personal, pero no aborda la dimensión educativa ni ofrece herramientas relacionadas con la creación de apuntes, el estudio o el refuerzo del aprendizaje. A diferencia de WrapItUp Planner, Todoist se limita a la organización de tareas y no integra funcionalidades académicas ni colaborativas.

Finalmente está Notion [3], una plataforma que combina notas, bases de datos, tareas y espacios colaborativos en un único entorno. Su flexibilidad permite adaptarla a múltiples usos, incluido el académico, aunque no está específicamente diseñada para estudiantes. Notion no ofrece herramientas nativas orientadas al refuerzo del aprendizaje, como la generación automática de *quizzes*, y tampoco integra un calendario estructurado dentro de un flujo de estudio. Su potencia radica en la personalización, pero requiere un mayor esfuerzo por parte del usuario para configurar su espacio de trabajo, mientras que WrapItUp Planner ofrece una experiencia más directa y guiada con el fin de optimizar el estudio.

Todas estas plataformas permiten comprender el contexto en el que se sitúa WrapItUp Planner, un entorno que combina organización, creación de apuntes, colaboración y refuerzo del aprendizaje en una sola plataforma. Su principal diferencia con otros servicios estaría en su enfoque académico y en la integración equilibrada entre herramientas manuales y funciones asistidas por IA. De este modo, WrapItUp Planner ofrece una experiencia unificada que otras aplicaciones solo cubren parcialmente.

1.3 Objetivos

Desde un inicio, el desarrollo de WrapItUp Planner tuvo como objetivo construir una plataforma web colaborativa que centralizara diferentes herramientas orientadas a la organización académica y al apoyo al estudio. Para ello, se desarrollaron funcionalidades enfocadas en la gestión de tareas, organización de apuntes, la posibilidad de compartir contenido entre usuarios e integración de herramientas basadas en inteligencia artificial, buscando ofrecer una experiencia unificada y práctica para estudiantes.

1.3.1 Objetivos funcionales

El objetivo principal de WrapItUp Planner es proporcionar una plataforma colaborativa donde los estudiantes puedan compartir y organizar sus materiales de estudio. Los usuarios de la aplicación se dividen en tres categorías: usuarios no registrados, usuarios registrados y administradores, cada una con funcionalidades y características personalizadas. Las funciones principales de la aplicación inclu-

yen la gestión de cuentas, el acceso a resúmenes compartidos (lo cuáles pueden ser generados por IA), un calendario interactivo con listas de tareas y la comunicación entre usuarios a través de una sección de comentarios con moderación adecuada.

Con el fin de facilitar la comprensión del sistema, se ha optado por agrupar sus funcionalidades en los siguientes módulos funcionales:

Gestión académica y planificación personal

Los usuarios registrados podrán utilizar un calendario interactivo en el que crear eventos de varios días, además de gestionar múltiples tareas diarias en formato de lista. Asimismo, dispondrán de una visualización en forma de mapa de calor que refleja la carga de trabajo diaria a lo largo del mes.

Gestión de notas y contenidos

Los usuarios registrados podrán crear y editar sus propias notas, organizándolas por categorías. Además, tendrán la posibilidad de subir su material de estudio al sistema, lo que permitirá la generación automática mediante inteligencia artificial de un resumen y un cuestionario asociado.

Aprendizaje y seguimiento del progreso

Tanto usuarios registrados como no registrados podrán acceder a las notas, si tienen permisos, y completar cuestionarios interactivos de opción múltiple. Adicionalmente, los usuarios registrados podrán visualizar gráficos de líneas que representan su progreso en los resultados de dichos cuestionarios.

Interacción social y colaboración

Los usuarios registrados podrán interactuar mediante comentarios en las páginas de los resúmenes, siempre que sean propietarios de las notas o tengan acceso a ellas. También podrán reportar comentarios que consideren inapropiados.

Administración y moderación

Los administradores podrán gestionar los reportes enviados por los usuarios y, en caso necesario, bloquear cuentas para garantizar el correcto uso de la plataforma.

1.3.2 Objetivos técnicos

WrapItUp Planner fue planteada como una aplicación web tipo SPA desarrollada con Angular en el *frontend* [4] y Spring Boot en el *backend* [5], que utiliza una base de datos SQL para el almacenamiento de datos [6]. El sistema expone una API REST documentada con OpenAPI [7] e integra inteligencia artificial mediante la API de OpenAI GPT-4o [8] para generar resúmenes y cuestionarios a partir del material de estudio del usuario. El desarrollo se gestiona con Git y GitHub [9], siguiendo un proceso iterativo con integración y despliegue continuo mediante GitHub Actions [10].

- **Frontend:** Desarrollo de una SPA con Angular para la interfaz de usuario y la gestión de interacciones.
- **Backend:** Implementación de una API REST con Spring Boot dentro de un proyecto Maven, encargada de la lógica de negocio.
- **Base de datos:** Uso de MySQL como sistema de gestión de datos.
- **Documentación de API:** Generación automática de documentación mediante OpenAPI.
- **Testing:** Implementación de pruebas unitarias y de integración para asegurar la calidad del software.
- **Calidad de código:** Análisis estático del código mediante SonarQube [11].
- **CI/CD:** Automatización de integración y despliegue continuo mediante GitHub Actions.
- **contenedorización:** Uso de Docker [12] para empaquetado y despliegue de la aplicación.
- **Inteligencia artificial:** Integración de la API de OpenAI GPT-4o para generación de contenido educativo.
- **Control de versiones:** Gestión del código mediante Git siguiendo la metodología GitHub Flow.

2

Metodología

El desarrollo de este proyecto ha sido planificado mediante múltiples fases, las cuales permitieron estructurar de forma progresiva la planificación, implementación y presentación del proyecto. Cada etapa aborda objetivos específicos, desde la definición inicial de funcionalidades y configuración del entorno de trabajo, hasta el desarrollo completo de la aplicación, la documentación final y la defensa del proyecto. Las fases definidas para la realización del proyecto son las siguientes:

- **Fase 1 - Definición de funcionalidades y pantallas.** En esta fase se definen las funcionalidades principales de la aplicación, las pantallas, los roles de usuario y las entidades de la base de datos. También se clasifican las funcionalidades en niveles básico, intermedio y avanzado.
- **Fase 2 - Repositorio, pruebas e integración continua.** Se crea el repositorio Git y se configuran las tecnologías principales. Se establece la conexión entre cliente, servidor y base de datos, junto con las primeras pruebas automáticas y la integración continua.
- **Fase 3 - Versión 0.1 - Funcionalidad básica y Docker.** Se desarrolla la funcionalidad básica de la aplicación, se añaden pruebas automatizadas y se implementa Docker para la contenedorización y la entrega continua. Se publica la versión 0.1.
- **Fase 4 - Versión 0.2 - Funcionalidad intermedia.** Se amplía la funcionalidad de la aplicación hasta un nivel intermedio, incorporando nuevas características y pruebas automatizadas. Se publica la versión 0.2.

- **Fase 5 - Versión 1.0 - Funcionalidad avanzada.** Se completa el desarrollo de la funcionalidad avanzada y se realiza el lanzamiento de la versión 1.0 final del sistema.
- **Fase 6 - Memoria.** Se redacta el informe final del proyecto, incluyendo la documentación completa del sistema y del proceso de desarrollo.
- **Fase 7 - Defensa.** Se realiza la preparación y presentación de la defensa final del proyecto.

La planificación temporal del proyecto se ha estructurado de forma que cada una de las fases descritas anteriormente cuente con unos objetivos y un periodo de ejecución definidos. Con el fin de facilitar la comprensión de dicha planificación, en la Tabla 2.1 se resumen las distintas fases junto con sus fechas de inicio y finalización.

Fase	Descripción	Inicio	Fin
1	Definición de funcionalidades y pantallas	28/07	15/09
2	Repositorio, pruebas e integración continua	16/09	15/10
3	Versión 0.1 - Funcionalidad básica y Docker	16/10	20/12
4	Versión 0.2 - Funcionalidad intermedia	20/12	18/03
5	Versión 1.0 - Funcionalidad avanzada	19/03	16/05
6	Memoria	17/05	01/06
7	Defensa	02/06	23/06

Tabla 2.1: Planificación temporal del proyecto

De manera complementaria, la Figura 2.1 muestra el diagrama de Gantt del proyecto, permitiendo visualizar de forma gráfica la distribución temporal de las distintas etapas y su secuenciación a lo largo del desarrollo.

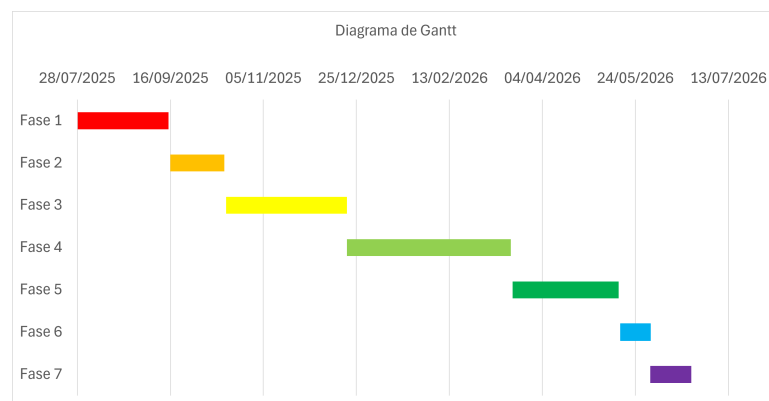


Figura 2.1: Diagrama de Gantt

3

Descripción funcional

Debido a la metodología de desarrollo empleada durante el proyecto, el planteamiento e implementación de las funcionalidades de la aplicación se dividió en tres etapas principales: funcionalidades básicas, funcionalidades intermedias y funcionalidades avanzadas. Esta organización permitió establecer una progresión clara en el desarrollo del sistema, comenzando por aquellas características esenciales para garantizar el funcionamiento mínimo de la plataforma y evolucionando así hacia funcionalidades más complejas y orientadas a mejorar la experiencia de usuario y la administración del sistema.

La división por etapas también facilitó la validación incremental de la aplicación, permitiendo comprobar el correcto funcionamiento de cada conjunto de características antes de avanzar hacia una nueva fase de desarrollo. De esta forma, las funcionalidades descritas a continuación se presentan siguiendo la misma estructura utilizada durante el proceso de implementación.

La descripción detallada de las funcionalidades, estructuradas por módulos funcionales, se encuentra disponible en el Anexo [A](#).

3.1 Descripción funcional

3.1.1 Funcionalidades básicas

Las funcionalidades básicas conforman el núcleo inicial del sistema, centrado en la gestión de usuarios, el acceso a contenido y el control de permisos. El sistema distingue tres roles principales: usuarios no registrados, usuarios registrados y administradores. Cada uno de estos roles con distintos niveles de acceso y capacidades dentro de la plataforma.

Los usuarios no registrados pueden registrarse e iniciar sesión en la aplicación, así como acceder a notas compartidas mediante enlaces siempre que estas no sean privadas. Por su parte, los usuarios registrados disponen de un perfil personal desde el cual pueden gestionar y visualizar sus propias notas, además de acceder a contenido compartido por otros usuarios cuando disponen de los permisos adecuados. Asimismo, cuentan con un sistema de comentarios asociado a las notas que permite la interacción entre usuarios.

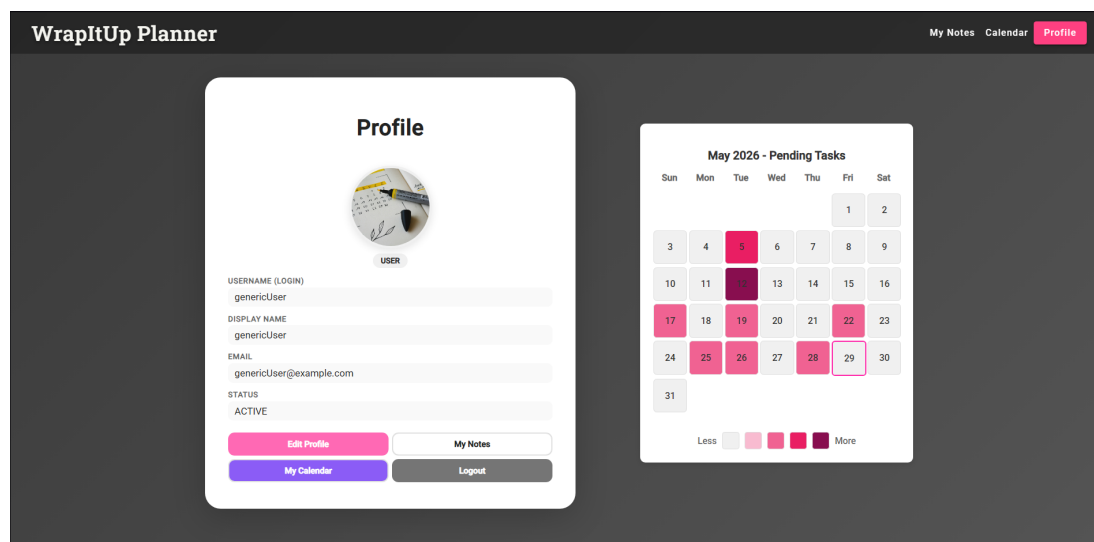


Figura 3.1: Perfil de usuario

Como se puede observar en la Figura 3.1, el perfil de usuario centraliza la información personal y el acceso a las distintas funcionalidades del sistema. De forma complementaria, el dashboard de notas, mostrado en la Figura 3.2, representa el panel principal de gestión de contenido, donde los usuarios pueden visualizar, filtrar y acceder a sus notas de forma estructurada.

Finalmente, los administradores disponen de permisos de gestión global sobre usuarios, notas y comentarios, incluyendo la eliminación de contenido y la supervisión general del sistema para garantizar su correcto funcionamiento.

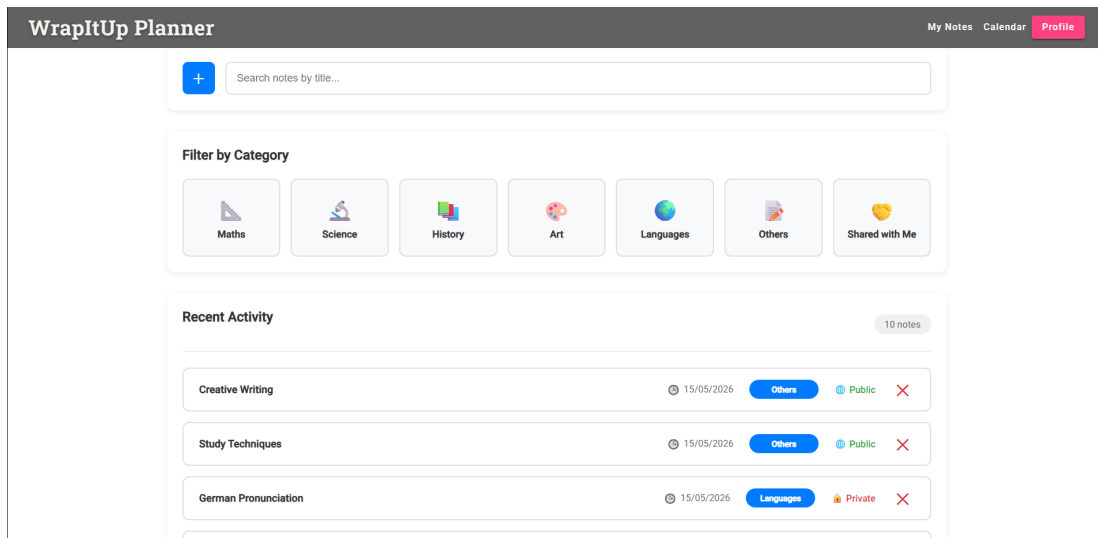


Figura 3.2: Dashboard de notas

3.1.2 Funcionalidades intermedias

Las funcionalidades intermedias amplían la capacidad del sistema en términos de organización personal, interacción y moderación, incorporando herramientas orientadas a la planificación del usuario y al control del contenido dentro de la plataforma.

En primer lugar, se introduce un calendario interactivo que permite la creación de eventos de uno o varios días, ofreciendo distintas vistas de visualización y acceso detallado a la información mediante la selección de fechas concretas, tal y como se muestra en la Figura 3.3.

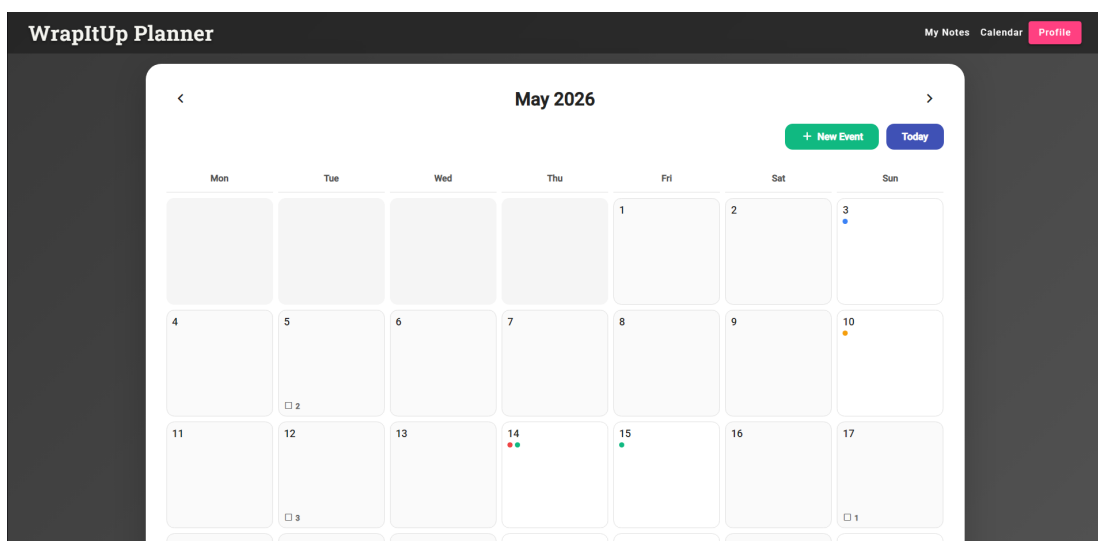


Figura 3.3: Vista mensual del calendario interactivo

Asimismo, se incorpora un sistema de tareas diarias tipo *to-do list*, orientado a mejorar la planificación personal del usuario y el seguimiento de actividades pendientes dentro de la plataforma. Tal y como se aprecia en la Figura 3.4, esta funcionalidad se integra con la vista diaria del calendario.

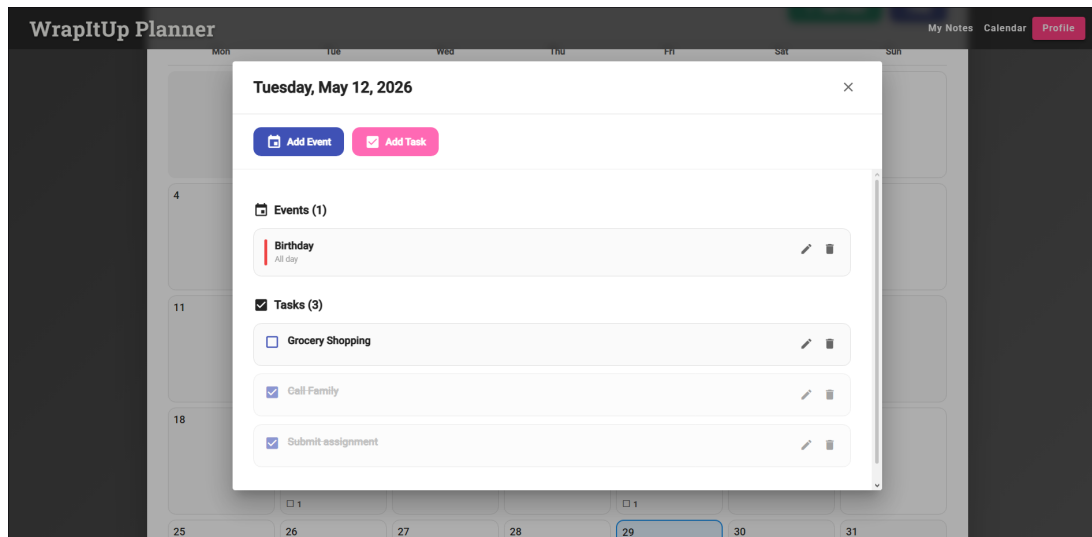


Figura 3.4: Vista diaria del calendario y tareas tipo to-do list

En el ámbito de la interacción y la seguridad, los usuarios disponen de un sistema para reportar comentarios inapropiados. Estos reportes son gestionados posteriormente por los administradores mediante un sistema de revisión de incidencias.

Desde el punto de vista de la administración del sistema, se implementa un panel específico de gestión de reportes que permite a los administradores revisar incidencias y bloquear usuarios cuando sea necesario, garantizando así el correcto uso de la plataforma. Esto lo podemos observar en la Figura 3.5.

Adicionalmente, se introduce un mapa de calor que representa de forma visual la actividad diaria del usuario en función de sus tareas pendientes. Como se muestra en la Figura 3.1, esta funcionalidad se integra dentro de la vista de perfil, en la zona derecha de la interfaz.

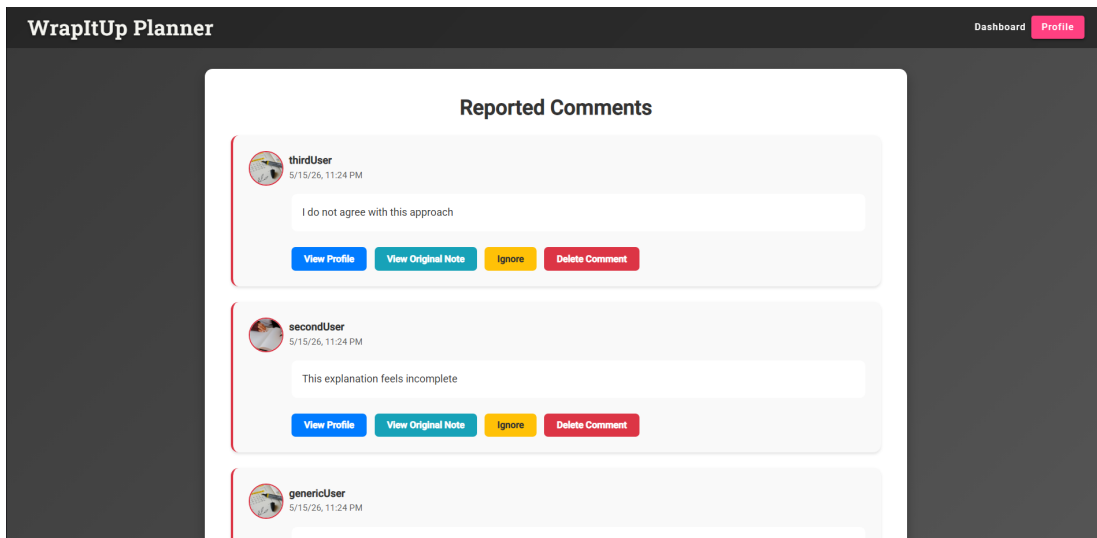


Figura 3.5: Panel de administración con gestión de reportes

3.1.3 Funcionalidades avanzadas

Las funcionalidades avanzadas introducen el uso de inteligencia artificial aplicada al aprendizaje, aportando el mayor valor añadido del sistema.

Como se muestra en la Figura 3.6, los usuarios pueden subir su material educativo, que será procesado automáticamente mediante un modelo de inteligencia artificial para generar un resumen estructurado y un conjunto de preguntas tipo test.

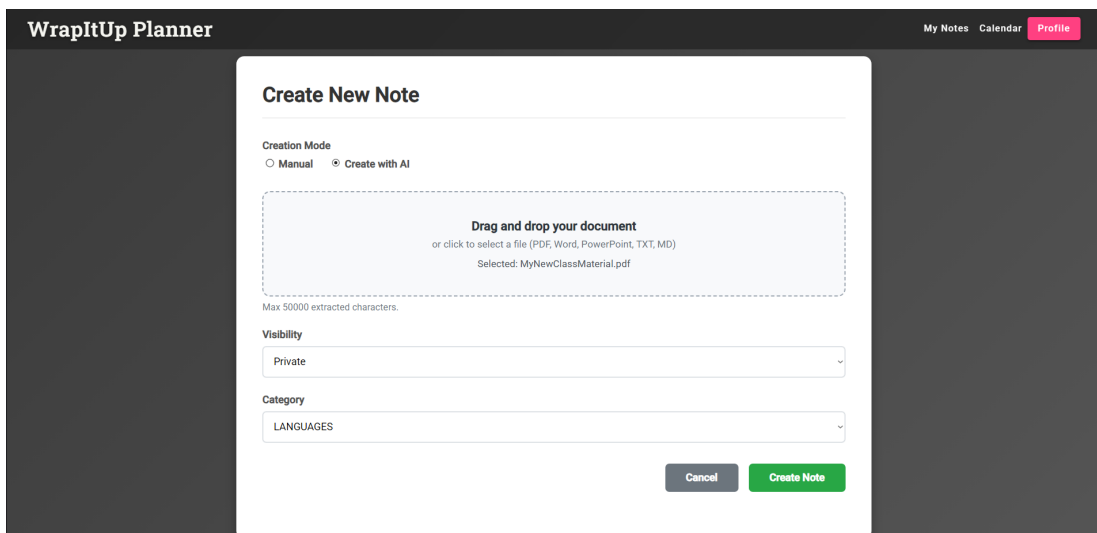


Figura 3.6: Pantalla de creación y subida de documentos

Estas preguntas permiten la realización de cuestionarios interactivos dentro de la plataforma, orientados a la autoevaluación del usuario, como se puede observar

en la Figura 3.7.

The screenshot shows the 'WrapItUp Planner' application with a 'Quiz' section. The interface includes a 'Hide Quiz' button and instructions to 'Answer all questions and submit to see your score.' Two questions are displayed:

1. What kind of triangle does the Pythagorean theorem apply to?

- ☐ Equilateral triangle
- ☒ Right triangle
- ☐ Isosceles triangle
- ☐ Scalene triangle

2. What relationship does the theorem describe?

- ☐ Angles and area
- ☒ Sides and the hypotenuse
- ☐ Perimeter and radius
- ☐ Base and height

Figura 3.7: Cuestionario tipo test generado mediante IA

Finalmente, el sistema registra los resultados obtenidos en estos cuestionarios y los representa mediante gráficos de evolución, permitiendo visualizar el progreso del usuario a lo largo del tiempo, tal y como se muestra en la Figura 3.8.

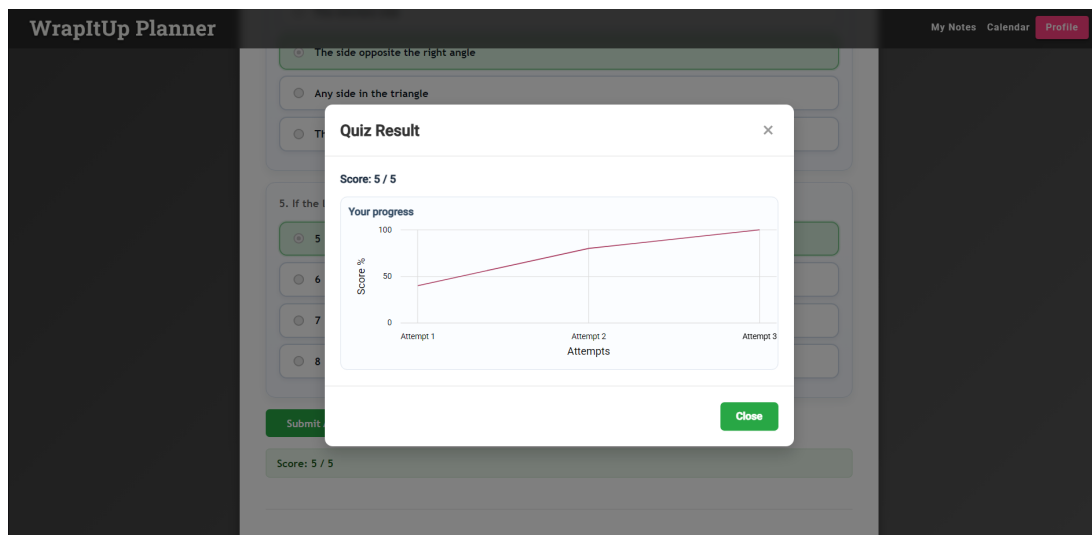


Figura 3.8: Gráfico de evolución del rendimiento del usuario

3.1.4 Recursos adicionales

Para la visualización de las funcionalidades, en el repositorio del proyecto se encuentran disponibles distintos vídeos organizados por etapas de desarrollo, en

los que se muestra la evolución del sistema y las funcionalidades implementadas en cada fase. Se dispone igualmente de un vídeo resumen con una demostración general de la aplicación, accesible a través del siguiente enlace de YouTube: <https://youtu.be/urIo-WkFKT8>.

El listado detallado de funcionalidades desarrolladas, donde se ofrece una descripción completa de cada una de ellas, puede consultarse en el Anexo [A](#).

Las instrucciones necesarias para la ejecución de la aplicación se encuentran disponibles en el Anexo [B](#).

4

Descripción Informática

Contenido del capítulo

4.1	Contexto	22
4.2	Tecnologías	23
4.2.1	Angular	23
4.2.2	Spring Boot	23
4.2.3	Java 21	24
4.2.4	Maven	24
4.2.5	ngx-charts	24
4.2.6	SQL y MySQL	24
4.2.7	Frameworks de pruebas	24
4.2.8	OpenAI GPT-4o	25
4.3	Herramientas	25
4.3.1	Visual Studio Code (VSCode)	25
4.3.2	Postman	25
4.3.3	GitHub	26
4.3.4	GitHub Projects	26
4.3.5	SonarQube / SonarCloud	26
4.3.6	JaCoCo	26
4.3.7	OpenAPI	26
4.4	Arquitectura	27

4.4.1	Despliegue	27
4.4.2	Modelo del dominio	27
4.4.3	API REST	28
4.4.4	Arquitectura del servidor	33
4.4.5	Arquitectura del cliente	34
4.5	Implementación	36
4.5.1	Vistas de calendario: <i>Month View</i> y <i>Day View</i>	36
4.5.2	Mapa de calor de tareas	37
4.5.3	Integración de inteligencia artificial	39
4.5.4	Procesamiento de documentos y extracción de texto	40
4.6	Control de calidad	41
4.6.1	Tipos de pruebas	42
4.6.2	Cobertura de pruebas	43
4.7	Despliegue	43
4.7.1	Empaquetado y distribución	43
4.7.2	Distribución	44
4.7.3	Despliegue remoto (URJC)	44
4.8	Proceso de desarrollo	45
4.8.1	Gestión de tareas	45
4.8.2	Git	46
4.8.3	Integración y entrega continua	47
4.8.4	Versionado	48

Este capítulo describe los aspectos técnicos del desarrollo de la aplicación. El código fuente está disponible en GitHub:

<https://github.com/codeurjc-students/2025-WrapItUp-Planner>

4.1 Contexto

WrapItUp Planner es una aplicación web tipo Single Page Application (SPA) desarrollada con Angular en el *frontend* y Spring Boot en el *backend*, dentro de un proyecto gestionado con Maven. Este tipo de arquitectura permite cargar una única página inicial y gestionar la navegación y las interacciones del usuario de forma dinámica, evitando recargas completas del navegador.

El sistema se estructura en tres componentes principales: el cliente, encargado de la interfaz de usuario; el servidor, responsable de la lógica de negocio y la exposición de una API REST documentada mediante OpenAPI; y la base de datos, un sistema SQL conectado al *backend* mediante MySQL Connector para la persistencia de datos.

La aplicación integra además inteligencia artificial mediante OpenAI GPT-4o, utilizada para generar automáticamente contenido de estudio a partir del material de estudio de los usuarios, como resúmenes y cuestionarios en formato estructurado.

El desarrollo se ha llevado a cabo siguiendo un enfoque iterativo e incremental, utilizando Git y GitHub como sistema de control de versiones, junto con procesos de integración y despliegue continuo automatizados mediante GitHub Actions.

Categoría	Tecnologías y Herramientas
Tipo	Aplicación web SPA con <i>backend</i> en Spring Boot y API REST
Tecnologías	Angular (<i>frontend</i>), Spring Boot (<i>backend</i>), Java 21, SQL, OpenAPI, OpenAI GPT-4o
Herramientas	VSCode con extensiones de Java y Spring Boot, Postman, GitHub, GitHub Projects, SonarQube Cloud
Control de calidad	Pruebas unitarias e integración con RestAssured, pruebas <i>frontend</i> con Karma y Jasmine, pruebas E2E con Selenium, cobertura con JaCoCo, análisis estático con SonarQube e integración continua con GitHub Actions
Despliegue	Integración y entrega continua mediante GitHub Actions, con uso de Docker para la contenerización y publicación de imágenes en Docker Hub
Proceso de desarrollo	Desarrollo iterativo e incremental basado en Git/-GitHub, con automatización de CI/CD mediante GitHub Actions

Tabla 4.1: Resumen general de las características del sistema Wrap-It-Up Planner

4.2 Tecnologías

A continuación, se describen las principales tecnologías utilizadas para el desarrollo y funcionamiento de la aplicación, indicando brevemente la función que desempeña cada una dentro del proyecto.

4.2.1 Angular

Angular es el framework utilizado para desarrollar el *frontend* de la aplicación. Gracias a su arquitectura basada en componentes, permite crear interfaces dinámicas, reutilizables y fáciles de mantener. En el proyecto, Angular se encarga de gestionar toda la interacción del usuario y de comunicarse con el *backend* mediante llamadas a la API REST. Disponible en <https://angular.io>

4.2.2 Spring Boot

Spring Boot es el framework utilizado para el desarrollo del *backend* de la aplicación. Facilita la creación de servicios REST y la organización de la lógica de negocio del sistema. En el proyecto, se utiliza para gestionar las peticiones del *frontend*, el acceso a la base de datos y las funcionalidades principales de la plataforma. Disponible en <https://spring.io/projects/spring-boot>

4.2.3 Java 21

Java 21 es el lenguaje empleado para implementar el *backend* junto con Spring Boot [13]. Se trata de una tecnología robusta y ampliamente utilizada en aplicaciones empresariales, lo que permite desarrollar un sistema mantenible y escalable. Disponible en <https://www.oracle.com/java/>

4.2.4 Maven

Maven es la herramienta utilizada para gestionar dependencias y automatizar la compilación del proyecto *backend* [14]. Además, facilita la integración de librerías externas y la generación de versiones consistentes de la aplicación. Disponible en <https://maven.apache.org>

4.2.5 ngx-charts

ngx-charts es una librería de visualización de datos para Angular basada en D3.js [15]. En este proyecto se utiliza para representar gráficamente la evolución del rendimiento del usuario en los cuestionarios, permitiendo una visualización clara e interactiva del progreso a lo largo del tiempo. Disponible en <https://swimlane.github.io/ngx-charts/>

4.2.6 SQL y MySQL

MySQL es el sistema de gestión de bases de datos utilizado en la aplicación. A través de consultas SQL, el *backend* realiza operaciones de almacenamiento y recuperación de información relacionada con usuarios, notas, comentarios, tareas y cuestionarios. Disponible en <https://www.mysql.com>

4.2.7 Frameworks de pruebas

Con el objetivo de garantizar la calidad del sistema, se emplearon distintas herramientas de testing. Mockito [16] y REST Assured [17] se utilizaron para pruebas del *backend*, mientras que Karma [18] y Jasmine [19] permitieron realizar pruebas de los componentes del *frontend* en Angular. Además, Selenium [20] se utilizó para la ejecución de pruebas end-to-end, simulando el comportamiento real de los usuarios sobre la aplicación. Disponibles en:

- Mockito: <https://site.mockito.org>

- REST Assured: <https://rest-assured.io/>
- Karma: <https://karma-runner.github.io/latest/index.html>
- Jasmine: <https://jasmine.github.io>
- Selenium: <https://www.selenium.dev>

4.2.8 OpenAI GPT-4o

GPT-4o es el modelo de inteligencia artificial integrado en la aplicación para procesar automáticamente el material subido por los usuarios. A partir de dicho contenido, el sistema genera resúmenes y preguntas tipo test utilizadas posteriormente en los cuestionarios interactivos. Disponible en <https://openai.com>

4.3 Herramientas

A continuación, se describen las principales herramientas empleadas durante el desarrollo del proyecto, utilizadas tanto para la implementación como para la gestión, prueba y aseguramiento de la calidad del sistema.

4.3.1 Visual Studio Code (VSCode)

Visual Studio Code es el editor de código utilizado para el desarrollo de la aplicación, tanto en el *frontend* con Angular como en el *backend* con Java y Spring Boot [21]. Su soporte para extensiones permite integrar herramientas específicas como Angular Language Service o soporte para Java, lo que mejora la productividad y facilita el desarrollo. Disponible en <https://code.visualstudio.com>

4.3.2 Postman

Postman es una herramienta utilizada para la prueba de APIs REST. Permite enviar solicitudes HTTP, analizar respuestas y validar el correcto funcionamiento de los endpoints del *backend* de forma sencilla y visual [22]. Disponible en <https://www.postman.com>

4.3.3 GitHub

GitHub es la plataforma utilizada para el control de versiones y la gestión del código fuente. En el proyecto se emplea para organizar el desarrollo mediante ramas y facilitar el trabajo colaborativo. Además, se integra con GitHub Actions para automatizar procesos de integración continua, incluyendo compilación y pruebas automáticas. Disponible en <https://github.com>

4.3.4 GitHub Projects

GitHub Projects se utiliza como herramienta de gestión de tareas y planificación del desarrollo. Permite organizar el trabajo mediante tableros, facilitando el seguimiento del progreso del proyecto y la asignación de tareas. Disponible en <https://github.com/features/project-management>

4.3.5 SonarQube / SonarCloud

SonarQube y SonarCloud son herramientas de análisis estático de código utilizadas para garantizar la calidad del software [11, 23]. Permiten detectar errores, vulnerabilidades y malas prácticas en el código fuente. En el proyecto se integran dentro del flujo de trabajo de GitHub Actions para automatizar las revisiones de calidad y mantener un código más limpio y mantenible. Disponible en <https://www.sonarqube.org> y <https://sonarcloud.io>

4.3.6 JaCoCo

JaCoCo es una herramienta de cobertura de código utilizada en proyectos Java. Permite medir qué partes del *backend* han sido ejecutadas durante las pruebas, generando informes que ayudan a evaluar y mejorar la calidad y cobertura de los tests [24]. Disponible en <https://www.jacoco.org>

4.3.7 OpenAPI

OpenAPI es una especificación utilizada para documentar automáticamente APIs REST. En el proyecto se emplea para generar documentación clara y estructurada de los endpoints del *backend*, facilitando su comprensión y uso. Disponible en <https://openapi-generator.tech>

4.4 Arquitectura

4.4.1 Despliegue

La aplicación sigue una arquitectura de despliegue en tres capas, donde el cliente, el servidor y la base de datos se ejecutan como procesos independientes y se comunican mediante protocolos estándar.

- **Frontend Angular:** se ejecuta en un servidor web y proporciona la interfaz de usuario. Se comunica con el *backend* mediante peticiones HTTP/HTTPS a la API REST.
- **Backend Spring Boot:** expone la API REST de la aplicación, implementa la lógica de negocio y gestiona el acceso a los datos mediante Spring Data JPA.
- **Base de datos MySQL:** almacena de forma persistente la información de la aplicación. El acceso a la base de datos se realiza exclusivamente desde el *backend* mediante el conector MySQL Connector.

La comunicación entre el *frontend* y el *backend* se realiza a través del protocolo HTTP/HTTPS utilizando mensajes en formato JSON. Por su parte, el *backend* establece una conexión directa con la base de datos mediante el protocolo nativo de MySQL. Esta separación de responsabilidades favorece la modularidad, el mantenimiento y la escalabilidad de la aplicación.

4.4.2 Modelo del dominio

El modelo de dominio mostrado en la Figura 4.1 representa las principales entidades persistentes de la aplicación y las relaciones existentes entre ellas, proporcionando una visión general de la estructura conceptual del sistema.

La entidad `UserModel` constituye el elemento central del modelo, ya que representa a los usuarios registrados de la plataforma. Cada usuario puede crear y gestionar múltiples notas (`Note`), comentarios (`Comment`), eventos de calendario (`CalendarEvent`) y tareas de calendario (`CalendarTask`).

Las notas almacenan el contenido generado por los usuarios y pueden contener múltiples comentarios asociados, permitiendo la interacción entre usuarios sobre dicho contenido. Por su parte, la entidad `QuizScore` registra los resultados obtenidos en los cuestionarios generados a partir de una nota, estableciendo una relación tanto con el usuario que realiza el cuestionario como con la nota sobre la que se evalúa.

Finalmente, las entidades **CalendarEvent** y **CalendarTask** permiten gestionar la planificación personal de cada usuario mediante la creación de eventos y tareas asociadas a su cuenta.

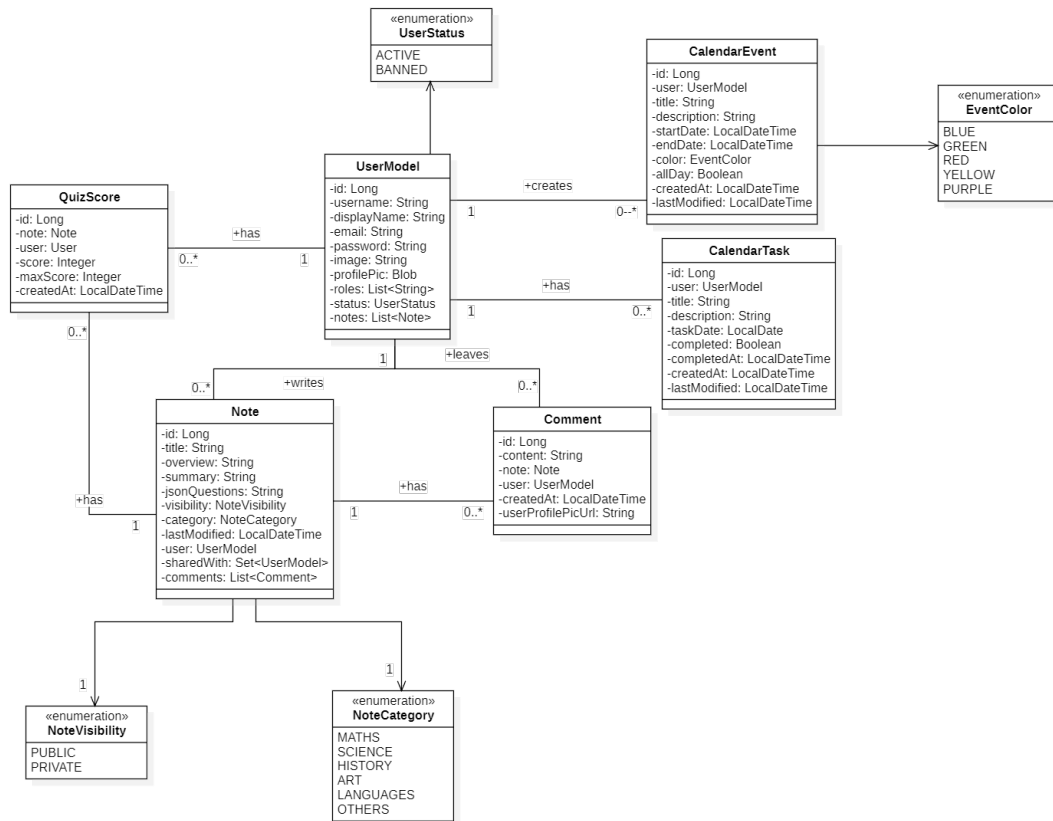


Figura 4.1: Modelo de dominio del sistema

4.4.3 API REST

La comunicación entre el cliente y el servidor se realiza mediante una API REST estructurada en diferentes recursos. Cada recurso encapsula una funcionalidad específica del sistema y se representa mediante uno o varios DTOs en formato JSON. A continuación, se presenta una descripción de alto nivel de los recursos disponibles y de las operaciones expuestas por cada uno de ellos a través de los distintos métodos HTTP.

Recurso Users

```
{
  "id": "integer(int64)",
  "username": "string",
  "displayName": "string",
  "email": "string",
  "password": "string(writeOnly)",
  "image": "string",
  "roles": "array<string>",
  "status": "string(enum: ACTIVE | BANNED)"
}
```

Operaciones

- **GET** `/api/v1/users` - Obtiene el usuario autenticado.
- **GET** `/api/v1/users/{id}` - Obtiene la informacion de un usuario por su identificador.
- **GET** `/api/v1/users/profile-image/{userId}` - Obtiene la imagen de perfil de un usuario.
- **POST** `/api/v1/users/{userId}/ban` - Bloquea de la plataforma al usuario indicado.
- **POST** `/api/v1/users/{userId}/unban` - Desbloquea al usuario indicado.
- **POST** `/api/v1/users/upload-image` - Sube una imagen de perfil del usuario.
- **PUT** `/api/v1/users` - Actualiza los datos del usuario autenticado.

Recurso Notes

```
{
  "id": "integer(int64)",
  "title": "string",
  "overview": "string",
  "summary": "string",
  "jsonQuestions": "string",
  "visibility": "string(enum: PUBLIC | PRIVATE)",
  "category": "string(enum: MATHS | SCIENCE | HISTORY | ART |
    LANGUAGES | OTHERS)",
  "lastModified": "string(date-time)",
  "userId": "integer(int64)",
  "sharedWithUserIds": "array<integer(int64)>",
  "comments": "array<CommentDTO>"
}
```

Operaciones

- **GET** `/api/v1/notes` - Obtiene notas recientes con paginacion y filtros.
- **GET** `/api/v1/notes/{id}` - Obtiene una nota por su identificador.
- **GET** `/api/v1/notes/shared` - Obtiene notas compartidas con el usuario.
- **POST** `/api/v1/notes` - Crea una nueva nota.
- **POST** `/api/v1/notes/{id}/ai/questions` - Genera preguntas con IA a partir de un archivo.
- **POST** `/api/v1/notes/ai` - Crea una nota con IA a partir de un archivo.
- **PUT** `/api/v1/notes/{id}` - Actualiza una nota existente.
- **PUT** `/api/v1/notes/{id}/share` - Comparte una nota con un usuario por username.
- **DELETE** `/api/v1/notes/{id}` - Elimina una nota existente.

Recurso Comments

```
{
  "id": "integer(int64)",
  "content": "string",
  "noteId": "integer(int64)",
  "userId": "integer(int64)",
  "username": "string",
  "displayName": "string",
  "userProfilePicUrl": "string",
  "createdAt": "string(date-time)",
  "reported": "boolean"
}
```

Operaciones (Usuario)

- **GET** `/api/v1/notes/{noteId}/comments` - Obtiene comentarios de una nota con paginacion.
- **POST** `/api/v1/notes/{noteId}/comments` - Crea un comentario en una nota.
- **POST** `/api/v1/notes/{noteId}/comments/{commentId}/report` - Reporta un comentario.
- **DELETE** `/api/v1/notes/{noteId}/comments/{commentId}` - Elimina un comentario.

Operaciones (Admin)

- **GET** `/api/v1/admin/reported-comments` - Obtiene comentarios reportados con paginacion.
- **POST** `/api/v1/admin/reported-comments/{commentId}/unreport` - Quita el reporte a un comentario.
- **DELETE** `/api/v1/admin/reported-comments/{commentId}` - Elimina un comentario reportado.

Recurso CalendarTask

```
{
  "id": "integer(int64)",
  "userId": "integer(int64)",
  "title": "string",
  "description": "string",
  "taskDate": "string(date)",
  "completed": "boolean",
  "completedAt": "string(date-time)",
  "createdAt": "string(date-time)",
  "lastModified": "string(date-time)"
}
```

Operaciones

- **GET** `/api/v1/calendar/tasks` - Obtiene tareas con filtros por fecha y estado.
- **POST** `/api/v1/calendar/tasks` - Crea una tarea de calendario.
- **PUT** `/api/v1/calendar/tasks/{id}` - Actualiza una tarea.
- **PATCH** `/api/v1/calendar/tasks/{id}/toggle` - Alterna el estado completado de una tarea.
- **DELETE** `/api/v1/calendar/tasks/{id}` - Elimina una tarea.

Recurso CalendarEvent

```
{
  "id": "integer(int64)",
  "userId": "integer(int64)",
  "title": "string",
  "description": "string",
  "startDate": "string(date-time)",
  "endDate": "string(date-time)",
  "color": "string(enum: BLUE | GREEN | RED | YELLOW | PURPLE)",
  "colorHex": "string",
  "allDay": "boolean",
  "createdAt": "string(date-time)",
  "lastModified": "string(date-time)"
}
```

Operaciones

- **GET** /api/v1/calendar/events - Obtiene eventos por rango de fechas.
- **POST** /api/v1/calendar/events - Crea un evento.
- **PUT** /api/v1/calendar/events/{id} - Actualiza un evento.
- **DELETE** /api/v1/calendar/events/{id} - Elimina un evento.

Recurso CalendarView

Usa los DTO: **CalendarMonthDTO** y **CalendarDayDTO** (según el endpoint).

CalendarMonthDTO

```
{
  "year": "integer(int32)",
  "month": "integer(int32)",
  "days": "array<CalendarDaySummaryDTO>"
}
```

CalendarDaySummaryDTO

```
{
  "date": "string(date)",
  "eventColors": "array<string>",
  "totalEvents": "integer(int32)",
  "pendingTasks": "integer(int32)",
  "hasEvents": "boolean",
  "hasTasks": "boolean"
}
```

CalendarDayDTO

```
{
  "date": "string(date)",
  "events": "array<CalendarEventDTO>",
  "tasks": "array<CalendarTaskDTO>"
}
```

Operaciones

- **GET** `/api/v1/calendar/month/{year}/{month}` - Obtiene la vista mensual del calendario (CalendarMonthDTO).
- **GET** `/api/v1/calendar/day` - Obtiene la vista diaria por fecha (CalendarDayDTO).
- **GET** `/api/v1/calendar/day/{year}/{month}/{day}` - Obtiene la vista diaria por componentes de fecha (CalendarDayDTO).

Recurso QuizResult

```
{
  "quizScore": "integer(int32)",
  "quizMaxScore": "integer(int32)",
  "quizProgressPercentages": "array<number(double)>"
}
```

Operaciones

- **POST** `/api/v1/notes/{id}/quiz-results` - Registra resultados de un quiz asociado a la nota.

4.4.4 Arquitectura del servidor

La arquitectura del servidor mostrada en la Figura 4.2 representa la estructura interna del *backend* y su organización siguiendo un enfoque en capas. Esta separación divide el sistema en controladores, servicios y repositorios, lo que permite aislar las responsabilidades y mejorar la mantenibilidad del código.

Los controladores se encargan de gestionar las peticiones HTTP recibidas desde el cliente y de coordinar la respuesta correspondiente. La capa de servicios contiene la lógica de negocio de la aplicación, mientras que la capa de repositorios gestiona el acceso a los datos y la interacción con la base de datos.

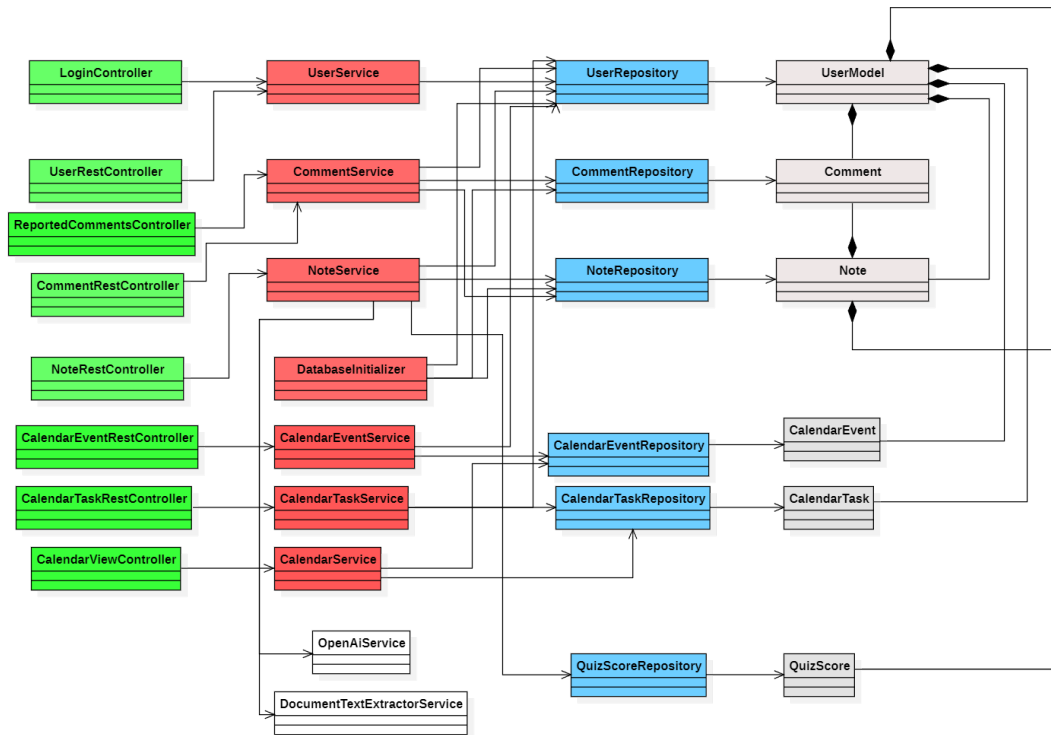


Figura 4.2: Arquitectura del servidor

4.4.5 Arquitectura del cliente

La arquitectura del cliente mostrada en la Figura 4.3 describe la estructura interna de la aplicación *frontend*, implementada como una Single Page Application (SPA) basada en Angular. Este enfoque implica que la aplicación carga una única página HTML y actualiza dinámicamente el contenido a medida que el usuario interactúa con ella, evitando recargas completas del navegador.

La arquitectura se organiza en componentes, templates y servicios. Los componentes gestionan la lógica de presentación y la interacción con el usuario, los templates definen la estructura visual de la interfaz y finalmente, los servicios se utilizan para la comunicación con el *backend*, centralizando las peticiones HTTP a la API REST.

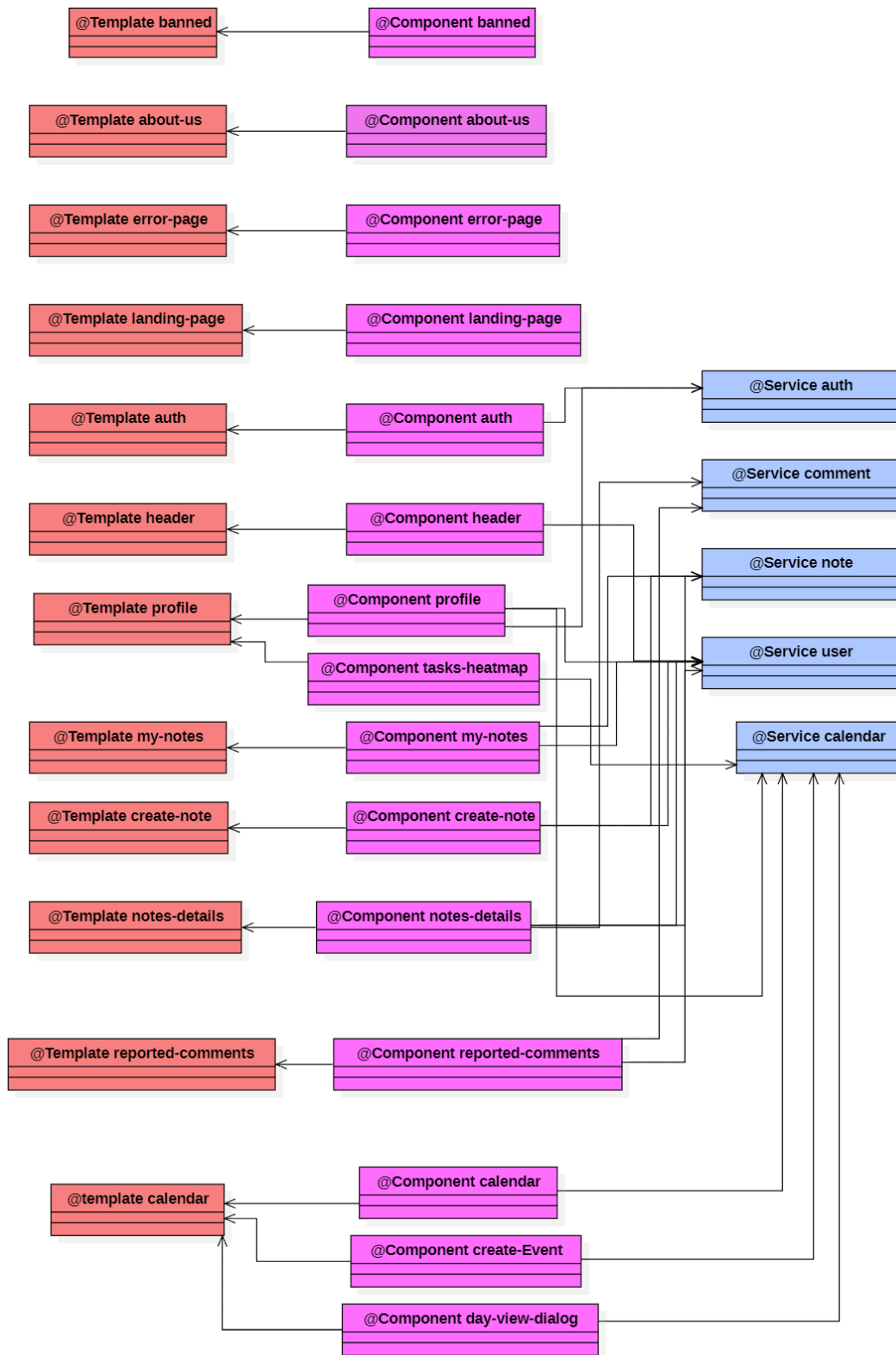


Figura 4.3: Arquitectura del cliente

4.5 Implementación

Esta sección describe los aspectos más relevantes e interesantes del proceso de implementación de la aplicación. En lugar de detallar exhaustivamente toda la aplicación, se presentan ejemplos representativos de la lógica de negocio y de la integración de tecnologías, con el objetivo de ilustrar cómo se han materializado los principales requisitos funcionales del sistema. Asimismo, se incluyen fragmentos de código relevantes para facilitar la comprensión de las soluciones adoptadas durante el desarrollo.

4.5.1 Vistas de calendario: *Month View* y *Day View*

El modulo de calendario implementa dos niveles de visualización: una vista mensual (*month view*) y una vista diaria (*day view*). La vista mensual ofrece un resumen por día (eventos y tareas pendientes), mientras que la vista diaria permite consultar el detalle completo de un día concreto. Ambos niveles se alimentan de endpoints dedicados del *backend* y se orquestan desde el *frontend* Angular mediante el servicio `CalendarService`.

Month View. El *frontend* consulta el endpoint mensual y construye una cuadrícula de semanas. Para ello, calcula el día de la semana en el que comienza el mes, rellena las celdas vacías iniciales y distribuye cada día dentro de una estructura `CalendarDaySummaryDTO`. De esta forma, la vista mensual mantiene el alineado de semanas y permite representar indicadores de eventos y tareas.

```

1  buildCalendarGrid(): void {
2      if (!this.monthData) return;
3
4      const firstDayOfMonth = new Date(this.currentYear, this.currentMonth - 1, 1);
5      let firstDayWeekday = firstDayOfMonth.getDay();
6      firstDayWeekday = firstDayWeekday === 0 ? 7 : firstDayWeekday;
7
8      const daysInMonth = new Date(this.currentYear, this.currentMonth,
9          0).getDate();
10
11     this.calendarDays = [];
12     let week: (CalendarDaySummaryDTO | null)[] = [];
13
14     for (let i = 1; i < firstDayWeekday; i++) {
15         week.push(null);
16     }
17
18     for (let day = 1; day <= daysInMonth; day++) {
19         const dateStr =
20             `${this.currentYear}-${String(this.currentMonth).padStart(2,
21                 '0')}-${String(day).padStart(2, '0')}`;
22         const daySummary = this.monthData.days.find(d => d.date === dateStr);
23
24         week.push(daySummary || {
25             date: dateStr,
26             eventColors: [],
27             totalEvents: 0,
28             pendingTasks: 0
29         });
30     }
31 }

```

```

27
28     if (week.length === 7) {
29         this.calendarDays.push(week);
30         week = [];
31     }
32 }
33
34 while (week.length > 0 && week.length < 7) {
35     week.push(null);
36 }
37 if (week.length > 0) {
38     this.calendarDays.push(week);
39 }
40 }

```

Código 4.1: Generacion de la cuadrícula mensual

Day View. La vista diaria se abre desde la vista mensual al seleccionar un día. Se utiliza un *modal* que recibe la fecha seleccionada y consulta el endpoint diario para obtener el detalle (eventos y tareas del día). Al cerrar el *modal*, se recarga el mes para reflejar cambios.

```

1  openDayView(daySummary: CalendarDaySummaryDTO | null): void {
2      if (!daySummary) return;
3
4      const [year, month, day] = daySummary.date.split('-').map(Number);
5      const date = new Date(year, month - 1, day);
6
7      const dialogRef = this.dialog.open(DayViewDialogComponent, {
8          width: '1000px',
9          maxWidth: '95vw',
10         maxHeight: '90vh',
11         data: {
12             year: date.getFullYear(),
13             month: date.getMonth() + 1,
14             day: date.getDate(),
15             date: daySummary.date
16         }
17     });
18
19     dialogRef.afterClosed().subscribe(() => {
20         this.loadMonth();
21     });
22 }

```

Código 4.2: Apertura de la vista diaria

4.5.2 Mapa de calor de tareas

Una funcionalidad interesante del módulo del calendario es la visualización de la carga de trabajo diaria mediante un mapa de calor. Este componente, implementado en Angular, proporciona una representación visual del número de tareas pendientes por día dentro de un mes, el actual.

Para su implementación, se reutiliza el endpoint del calendario que devuelve la información mensual agregada (*CalendarMonthDTO*). A partir de estos datos,

el componente calcula el número máximo de tareas pendientes del mes, lo que permite normalizar la escala de colores.

La intensidad del color de cada día se determina en función de la proporción de tareas pendientes respecto al valor máximo del mes. De este modo, los días con mayor carga de trabajo se muestran con tonos más oscuros, mientras que aquellos con menor actividad aparecen con colores más claros, logrando una visualización adaptativa independientemente del volumen total de tareas.

Además, el componente incluye interacción adicional mediante *hover*, que permite consultar información detallada de cada día, como la fecha y el número de tareas pendientes.

A continuación, se muestra un fragmento representativo de la lógica utilizada para generar el mapa de calor en el componente Angular:

```

1
2 generateWeeks(): void {
3   if (!this.monthData) return;
4
5   const firstDay = new Date(this.monthData.days[0].date);
6   const startDayOfWeek = firstDay.getDay();
7
8   let week: (CalendarDaySummaryDTO | null)[] = [];
9
10  for (let i = 0; i < startDayOfWeek; i++) {
11    week.push(null);
12  }
13
14  this.monthData.days.forEach((day) => {
15    week.push(day);
16    if (week.length === 7) {
17      this.weeks.push(week);
18      week = [];
19    }
20  });
21
22  if (week.length > 0) {
23    while (week.length < 7) {
24      week.push(null);
25    }
26    this.weeks.push(week);
27  }
28 }
```

Código 4.3: Generación de semanas en el mapa de calor

```

1
2 getHeatmapColor(day: CalendarDaySummaryDTO | null): string {
3   if (!day || day.pendingTasks === 0) {
4     return '#f0f0f0';
5   }
6
7   const intensity = Math.min(day.pendingTasks / this.maxTasks, 1);
8
9   if (intensity <= 0.25) {
10    return '#f8bbd0';
11  } else if (intensity <= 0.5) {
12    return '#f06292';
13  } else if (intensity <= 0.75) {
14    return '#e91e63';
15  } else {
16    return '#880e4f';
17  }
18 }

```

Código 4.4: Cálculo del color del mapa de calor

4.5.3 Integración de inteligencia artificial

Un aspecto clave en la integración de este módulo ha sido el diseño de los *prompt* utilizados para la comunicación con el modelo de lenguaje. El objetivo principal no se ha limitado a obtener respuestas correctas desde el punto de vista semántico, sino a garantizar que la salida del modelo siga una estructura estricta y fácilmente procesable por el sistema.

En este sentido, se definió un *prompt* principal que obliga al modelo a devolver la información en formato JSON, incluyendo campos como el título, el resumen y un conjunto de preguntas tipo test. Dado que esta información se utiliza posteriormente para la generación de entidades dentro de la aplicación, resulta fundamental asegurar la consistencia del formato de salida, así como el cumplimiento de restricciones sobre la longitud de los distintos campos y el número de preguntas generadas.

Otro aspecto relevante ha sido la independencia del idioma. Aunque la interfaz de la aplicación está en inglés, los resúmenes generados no están restringidos a este idioma, ya que se instruye al modelo para que utilice el mismo idioma del documento de entrada, mejorando así la coherencia del contenido y la experiencia del usuario.

Asimismo, se ha diseñado un segundo *prompt* específico para el caso en el que el usuario desea generar únicamente preguntas tipo test a partir de una nota existente, sin necesidad de regenerar el resumen completo. Esta separación permite optimizar el uso del modelo y adaptar su comportamiento a distintos flujos dentro de la aplicación.

Toda esta lógica se implementa en el servicio responsable de la comunicación con la API de OpenAI, donde el método `buildPrompt` construye dinámicamente

el prompt a partir del texto de entrada del usuario y de las instrucciones necesarias para garantizar una salida estructurada.

A continuación, se muestra el fragmento principal responsable de la construcción del *prompt*:

```

1
2 private String buildPrompt(String text) {
3     return "Analyze the following text and create a summary.\n" +
4           "Return ONLY valid JSON with this exact structure:\n" +
5           "{\n" +
6           "  \"title\": \"Short title of the document\",\n" +
7           "  \"overview\": \"Brief summary in 2-3 sentences\",\n" +
8           "  \"completeSummary\": \"Detailed multi-paragraph summary around 4000\n" +
9           "    characters\",\n" +
10          "  \"jsonQuestions\": {\n" +
11          "    \"questions\": [\n" +
12          "      {\n" +
13          "        \"question\": \"Question text\",\n" +
14          "        \"options\": [\"Option A\", \"Option B\", \"Option C\", \"Option D\"],\n" +
15          "        \"correctOptionIndex\": 0\n" +
16          "      }\n" +
17          "    ]\n" +
18          "  }\n" +
19          "  Write the summary in the same language as the original text.\n" +
20          "  Do NOT use phrases like 'this document', 'the text', 'the file', or similar\n" +
21          "  meta-references.\n" +
22          "  Always refer directly to the topic, concepts, entities, events, formulas, or\n" +
23          "  procedures found in the content.\n" +
24          "  The completeSummary must be detail-oriented and substantive:\n" +
25          "  - Explain important mechanisms, steps, causes/effects, definitions, and examples\n" +
26          "    when present.\n" +
27          "  - Cover concrete details and key facts, not a generic overview.\n" +
28          "  - Avoid repeating the overview with slightly more words.\n" +
29          "  Generate between 5 and 10 multiple-choice questions.\n" +
30          "  Each question must include exactly 4 options and only one correct answer represented\n" +
31          "    by correctOptionIndex (0-3).\n" +
32          "  Limits: title max 100 characters, overview max " + maxOverviewChars +
33          "    characters, completeSummary max " + maxSummaryChars + " characters.\n" +
34          "  Text:\n" +
35          text;
36 }

```

Código 4.5: Construcción del prompt en OpenAIService

4.5.4 Procesamiento de documentos y extracción de texto

Uno de los componentes clave del sistema es el módulo encargado del procesamiento de documentos, cuya finalidad es transformar archivos en distintos formatos a texto plano. Este proceso es un paso previo esencial para la integración con el módulo de inteligencia artificial, ya que permite unificar la entrada de datos independientemente del formato original del documento.

Para implementar esta funcionalidad se ha desarrollado el servicio `DocumentTextExtractorService`, el cual actúa como punto de entrada único para la extracción de texto. Este servicio detecta automáticamente el tipo de archivo subido por el usuario y delega su procesamiento a un método específico

en función de su extensión, soportando formatos como *TXT*, *Markdown*, *PDF*, *DOCX* y *PPTX*.

La implementación se apoya en distintas librerías especializadas según el tipo de documento. Para la extracción de texto en archivos PDF se utiliza **Apache PDFBox**, mientras que los documentos de Word se procesan mediante **Apache POI (XWPF)**. Del mismo modo, las presentaciones de PowerPoint se gestionan utilizando el módulo **XSLF** de Apache POI. Estas librerías permiten acceder a la estructura interna de los documentos y extraer su contenido textual de forma robusta.

A continuación, se muestra como ejemplo el método encargado del procesamiento de archivos PDF:

```
1 private String readPdf(MultipartFile file) {
2     try (InputStream inputStream = file.getInputStream();
3         PDDocument document = PDDocument.load(inputStream)) {
4
5         PDFTextStripper stripper = new PDFTextStripper();
6         String text = stripper.getText(document).trim();
7
8         if (text.isBlank()) {
9             throw new IllegalArgumentException("PDF contains no readable text");
10        }
11
12        return text;
13    } catch (IOException e) {
14        throw new IllegalArgumentException("Unable to read PDF", e);
15    }
16 }
```

Código 4.6: Extracción de texto desde archivos PDF usando PDFBox

4.6 Control de calidad

Con el objetivo de garantizar la calidad y fiabilidad de la aplicación WrapItUp Planner, se ha definido una estrategia de pruebas automatizadas aplicada tanto al *backend* como al *frontend*. Esta estrategia permite verificar el correcto funcionamiento del sistema a nivel de servicios, integración entre capas y comportamiento de la interfaz de usuario.

En términos generales, todas las funcionalidades implementadas en la versión final del sistema han sido cubiertas al menos parcialmente por pruebas automáticas, asegurando así la validación de los principales flujos de negocio y casos de uso de la aplicación.

4.6.1 Tipos de pruebas

Pruebas del backend

En el *backend* se han definido distintos niveles de pruebas con el objetivo de validar la lógica de negocio, la integración con la base de datos y el correcto funcionamiento de la API REST.

- **Pruebas unitarias** (*unit*): validan los servicios de forma aislada mediante el uso de dobles de prueba (*mocks*), permitiendo comprobar su comportamiento sin depender del resto del sistema. Se han implementado utilizando **JUnit 5** y **Mockito**.
- **Pruebas de integración** (*integration*): levantan el contexto completo de Spring para verificar el correcto funcionamiento de la API REST y la interacción entre los distintos componentes del sistema. Estas pruebas se han realizado utilizando **RestAssured**.
- **Pruebas de sistema** (*system*): validan flujos completos del *backend*, combinando servicios y repositorios en un entorno controlado. Se utilizan **SpringBootTest** y transacciones para garantizar el aislamiento de los datos durante la ejecución de las pruebas.
- **Pruebas end-to-end web** (*e2e*): aunque se ejecutan a nivel de sistema completo, validan la interacción del *backend* con el *frontend* mediante la simulación de flujos de usuario a través de **Selenium WebDriver**.

Pruebas del frontend

En el *frontend* se han realizado pruebas orientadas a validar el comportamiento de los componentes y su integración con la API REST.

- **Pruebas de componentes**: verifican el comportamiento de los componentes de forma aislada, asegurando su correcta renderización, gestión de eventos e interacción con el usuario. Estas pruebas se han implementado utilizando **Karma** y **Jasmine** como herramientas principales.
- **Pruebas de servicios**: validan la lógica de los servicios del *frontend* y su interacción con la API REST, comprobando que las peticiones HTTP se construyen y gestionan correctamente, así como la correcta recepción y procesamiento de los datos.

4.6.2 Cobertura de pruebas

La cobertura de pruebas del sistema se ha monitorizado mediante distintas herramientas según el entorno. Para el *backend*, se ha utilizado **SonarCloud**, que permite realizar un análisis continuo de la calidad del código, incluyendo métricas de cobertura, mantenibilidad y posibles vulnerabilidades. Además, este análisis se ejecuta automáticamente en cada Pull Request dirigido a la rama *main*, lo que garantiza el cumplimiento de los estándares de calidad establecidos.

En el caso del *frontend*, la cobertura se ha medido mediante **Karma** junto con el *coverage reporter*, generando informes detallados sobre la ejecución de pruebas y el porcentaje de código cubierto en los distintos módulos de la aplicación.

Los resultados obtenidos se muestran en las Figuras 4.4 y 4.5, reflejando un nivel de cobertura acorde a los objetivos de calidad definidos durante el desarrollo del proyecto.

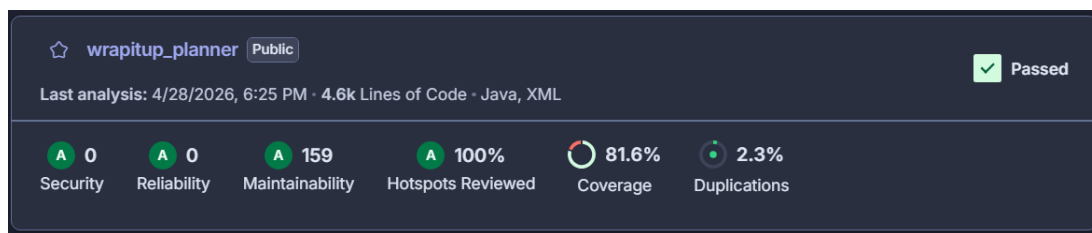


Figura 4.4: Cobertura y análisis estático del backend en SonarCloud

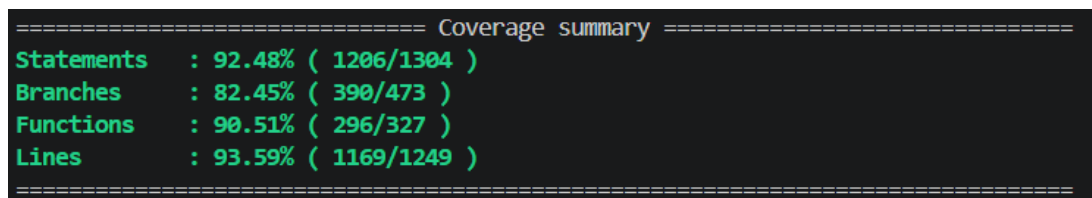


Figura 4.5: Cobertura de pruebas del frontend generada con Karma

4.7 Despliegue

4.7.1 Empaquetado y distribución

La aplicación se distribuye mediante **Docker**, lo que permite garantizar la portabilidad y la consistencia del entorno de ejecución en diferentes plataformas.

Imagen Docker

Se emplea una única **imagen Docker multi-stage** que integra tanto el *frontend* desarrollado en Angular como el *backend* implementado en Spring Boot. Este enfoque permite unificar el proceso de construcción y simplificar el despliegue de la aplicación.

Docker Compose

La orquestación de los distintos servicios del sistema se realiza mediante **Docker Compose**, que permite desplegar de forma conjunta el *frontend*, el *backend* y la base de datos MySQL. Esta configuración gestiona la comunicación entre contenedores, la definición de variables de entorno y el mapeo de puertos, incluyendo la exposición segura del servicio mediante HTTPS en el puerto 443.

4.7.2 Distribución

La aplicación se encuentra disponible públicamente a través de **DockerHub**, facilitando su despliegue en cualquier entorno compatible con Docker.

Repositorio: https://hub.docker.com/r/arturox2500/wrapitup_planner

Versiones disponibles:

- **latest:** versión más reciente en desarrollo.
- **dev:** versión de desarrollo generada automáticamente desde la rama principal.
- **0.1:** primera versión estable del sistema.
- **0.2:** segunda versión estable con mejoras incrementales.
- **1.0:** versión final del proyecto.

4.7.3 Despliegue remoto (URJC)

Durante el desarrollo del proyecto, la aplicación fue desplegada en una máquina remota proporcionada por la Universidad Rey Juan Carlos (URJC), con acceso mediante SSH. Este entorno cuenta con los permisos necesarios y la configuración de claves de acceso requeridas para la administración del sistema.

En dicho servidor se instalaron **Docker** y **Docker Compose**, permitiendo la ejecución de la aplicación junto con su base de datos en un entorno controlado.

Proxy inverso (Nginx)

Se utiliza **Nginx** como proxy inverso para la exposición segura de la aplicación, habilitando el acceso mediante HTTPS a través de un certificado SSL almacenado en `/home/vmuser/certs/`.

Este componente cumple las siguientes funciones:

- Terminación de conexiones HTTPS y gestión de certificados.
- Redirección automática del tráfico HTTP hacia HTTPS.
- Enrutamiento de las peticiones hacia el *backend* desarrollado en Spring Boot.

4.8 Proceso de desarrollo

Durante el desarrollo del proyecto se siguió una metodología de trabajo iterativa e incremental, alineada con los principios del Manifiesto Ágil [25]. El desarrollo se organizó en distintas fases que permitieron planificar, implementar y validar progresivamente las funcionalidades de la aplicación lo cuál facilitó la incorporación gradual de nuevas características, comenzando por los elementos esenciales del sistema y evolucionando así hacia funcionalidades más complejas. De esta manera, cada etapa permitió verificar el correcto funcionamiento de los componentes desarrollados antes de avanzar a la siguiente fase.

La metodología empleada se apoyó en algunas prácticas habituales de la Programación Extrema (XP) [26], como el desarrollo incremental, la integración continua y la realización de pruebas durante todo el proceso de implementación. Del mismo modo, la gestión del trabajo se inspiró en Kanban mediante la organización y seguimiento de tareas a lo largo de las diferentes fases del proyecto.

En los siguientes apartados se describen los mecanismos utilizados para la planificación y seguimiento de tareas, la estrategia de control de versiones adoptada mediante Git, los procesos automáticos de integración continua implementados durante el desarrollo y las técnicas empleadas para el versionado de la aplicación.

4.8.1 Gestión de tareas

Para la planificación y seguimiento de las tareas del proyecto se utilizó GitHub Projects, aprovechando su integración con GitHub Issues para centralizar la gestión del trabajo. Esta herramienta permitió mantener un registro organizado de

las funcionalidades pendientes de implementar, así como de los errores y mejoras identificados durante el desarrollo.

La gestión de tareas se realizó mediante un tablero visual inspirado en la metodología Kanban, estructurado en tres columnas principales: *To Do*, *In Progress* y *Done*. Cada tarea se representaba mediante una incidencia (*issue*) asociada al proyecto, facilitando el seguimiento de su estado a lo largo del ciclo de desarrollo. Este sistema permitió visualizar de forma rápida el trabajo pendiente, las tareas en curso y las funcionalidades ya completadas.

Además de las funcionalidades planificadas inicialmente, el tablero se utilizó para registrar incidencias detectadas durante las pruebas y el desarrollo de la aplicación. De esta forma, los errores identificados se incorporaban al flujo de trabajo como nuevas tareas pendientes, permitiendo su seguimiento y resolución de manera estructurada. El tablero empleado se encuentra dentro de la vista *Backlog* del proyecto en GitHub Projects.

La Figura 4.6 muestra el tablero utilizado durante el desarrollo, donde puede observarse la organización visual de las tareas según su estado.

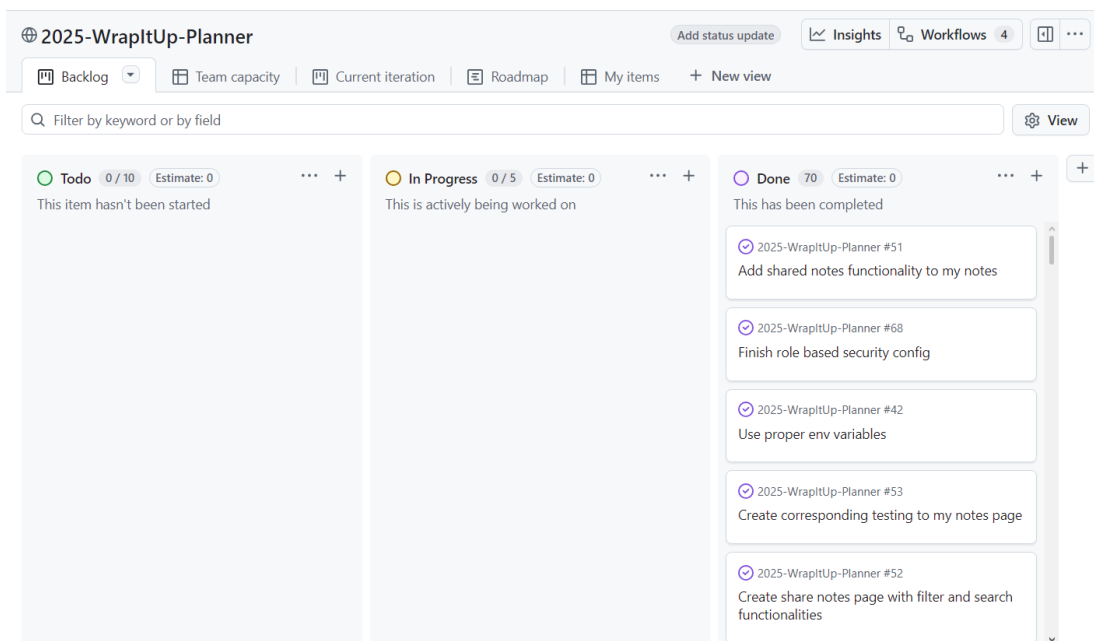


Figura 4.6: Tablero de gestión de tareas en GitHub Projects

4.8.2 Git

Durante el desarrollo del proyecto se ha utilizado Git como sistema de control de versiones, con el repositorio alojado en GitHub, permitiendo gestionar de forma eficiente el historial de cambios y facilitar el trabajo incremental sobre la

aplicación.

La estrategia de ramas adoptada se ha basado en un flujo de trabajo tipo GitHub Flow, donde la rama principal (*main*) se mantiene como la versión estable del proyecto, conteniendo únicamente código funcional que haya sido correctamente validado. El desarrollo de nuevas funcionalidades y correcciones se ha realizado en ramas independientes a partir de esta rama principal, siguiendo una convención de nomenclatura basada en *feature/* para nuevas funcionalidades y *fix/* para corrección de errores. Posteriormente, los cambios han sido integrados mediante *pull requests*, garantizando la revisión y validación del código antes de su incorporación al proyecto.

En cuanto al uso del repositorio, se han realizado 261 *commits* y se han gestionado 52 *pull requests* cerradas, lo que refleja el carácter iterativo del desarrollo y el uso continuo de integración de cambios durante todo el proyecto.

En la Figura 4.7 se muestra la evolución del número de *commits* a lo largo del desarrollo del proyecto.

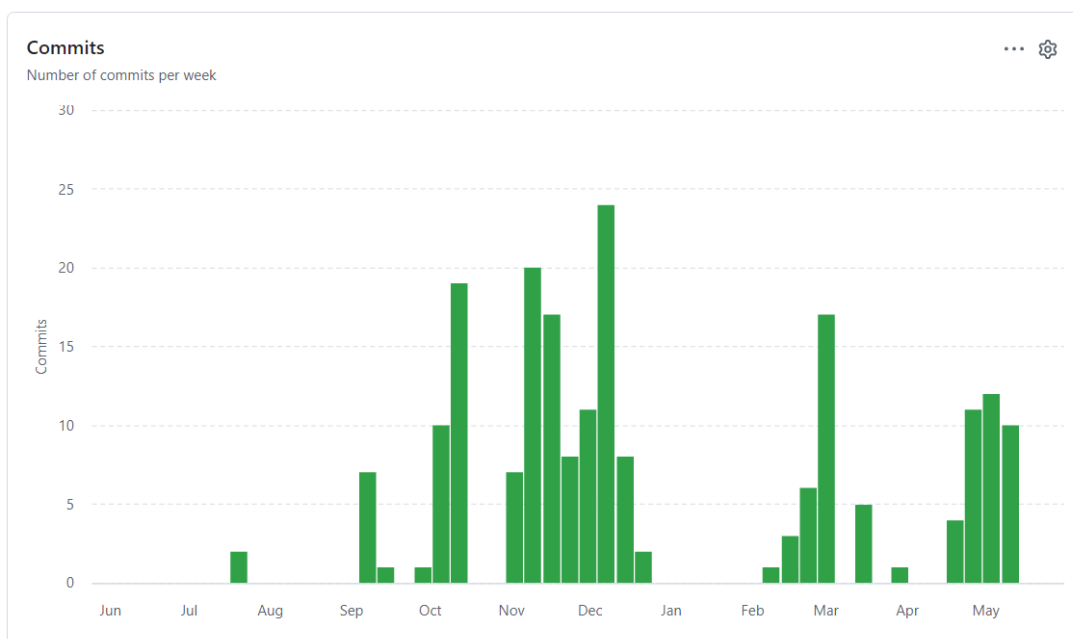


Figura 4.7: Evolución de commits del repositorio

4.8.3 Integración y entrega continua

Durante el desarrollo del proyecto se ha implementado un sistema de integración continua basado en GitHub Actions, con el objetivo de automatizar la validación del código y la generación Docker. En total, el sistema está compuesto por

cinco *workflows* que cubren tanto la fase de pruebas como la de construcción y publicación de la aplicación.

En primer lugar, se han definido dos *workflows* orientados a la ejecución de pruebas. El *workflow ci-basic* se ejecuta automáticamente en cada commit realizado sobre ramas de tipo *feature/** o *fix/**, ejecutando pruebas unitarias tanto del *backend* como del *frontend*, con el objetivo de detectar errores de forma temprana durante el desarrollo.

Por otro lado, el *workflow ci-full* se ejecuta cuando se abre una *pull request* hacia la rama principal (*main*). Este *workflow* ejecuta la batería completa de pruebas, incluyendo pruebas unitarias, de integración y de sistema. Además, incorpora un análisis estático de código mediante SonarCloud, permitiendo monitorizar la calidad del código y asegurar el cumplimiento de métricas mínimas, como una cobertura de pruebas superior al 80%.

Además, se han definido tres *workflows* adicionales encargados de la generación y publicación de imágenes Docker. El *workflow docker-main* se ejecuta automáticamente al integrarse cambios en la rama principal, generando una imagen etiquetada como *dev*. El *workflow docker-manual* permite generar manualmente una imagen de tipo *snapshot*, entendida como una versión que captura el estado del proyecto en un momento concreto, lo que resulta útil para pruebas intermedias. Finalmente, el *workflow docker-release* se ejecuta al crear una nueva release en GitHub, generando una imagen versionada según el número de versión del proyecto y actualizando la etiqueta *latest*.

4.8.4 Versionado

Para la gestión de versiones del proyecto se ha utilizado la convención *Semantic Versioning* (SemVer), un sistema de versionado basado en tres números de la forma *MAJOR.MINOR.PATCH*. Este esquema permite estructurar las versiones de manera clara: el incremento del número *MAJOR* indica cambios significativos o incompatibles, el número *MINOR* se incrementa al añadir nueva funcionalidad de forma compatible, y el número *PATCH* se utiliza para correcciones o mejoras menores.

En el contexto de este proyecto, el uso de SemVer ha permitido reflejar la evolución progresiva del sistema a lo largo de sus distintas fases de desarrollo. Durante la implementación de las funcionalidades básicas, intermedias y avanzadas se publicaron las versiones 0.1.0, 0.2.0 y 1.0.0 respectivamente.

La versión 0.1.0 se centró en la implementación de las funcionalidades básicas de gestión de notas y control de usuarios. Posteriormente, la versión 0.2.0 incorporó principalmente las funcionalidades relacionadas con el calendario y el panel de los administradores. Finalmente, la versión 1.0.0 integró el conjunto completo

de funcionalidades avanzadas, destacando especialmente aquellas basadas en inteligencia artificial, como la generación automática de contenido y cuestionarios a partir de material de estudio.

5

Conclusiones y trabajos futuros

5.1 Conclusiones

En este capítulo se presentan las conclusiones finales del proyecto, evaluando el grado de cumplimiento de los objetivos inicialmente establecidos, tanto desde el punto de vista funcional como técnico. Asimismo, se recoge una valoración personal del trabajo realizado y se proponen posibles líneas de mejora y evolución futura del sistema desarrollado.

5.1.1 Cumplimiento de objetivos

Respecto a los objetivos funcionales, todas las funcionalidades definidas durante las primeras fases del proyecto fueron implementadas de forma progresiva a lo largo de las distintas iteraciones de desarrollo. Inicialmente, el sistema se centró en la gestión de notas y usuarios, incorporando posteriormente funcionalidades más complejas como el calendario, la moderación de comentarios, la generación de contenido mediante inteligencia artificial y el seguimiento del progreso académico. La implementación de los módulos de notas y planificación personal en etapas diferentes dificultó inicialmente visualizar el alcance completo de la aplicación, pero la integración gradual de todas las funcionalidades permitió alcanzar la visión planteada al comienzo del proyecto y construir una plataforma cohesiva que cumple los requisitos definidos.

En cuanto a los objetivos técnicos, el proyecto ha logrado cumplir las me-

tas establecidas desde las primeras etapas de planificación. Se ha desarrollado una aplicación SPA utilizando Angular y Spring Boot, apoyada por una base de datos MySQL y una API REST documentada mediante OpenAPI. Asimismo, se han incorporado mecanismos de calidad como pruebas automatizadas, análisis estático de código y procesos de integración y despliegue continuo mediante GitHub Actions. Finalmente, la aplicación ha sido contenedorizada con Docker e integra modelos de inteligencia artificial para la generación de contenido académico, completando así los objetivos tecnológicos previstos para la versión final del sistema.

5.1.2 Valoración personal del proyecto

Este proyecto representa la culminación de los conocimientos y habilidades adquiridos a lo largo de mi formación universitaria. WrapItUp Planner es el resultado de cuatro años de aprendizaje y esfuerzo, en los que tanto los éxitos como las dificultades han contribuido a mi desarrollo como futuro profesional de la ingeniería del software.

El desarrollo de la aplicación durante este último año supuso reto constante. Ha sido un proceso exigente, repleto de momentos de incertidumbre, errores y problemas que resolver, pero también por la satisfacción de ver cómo cada etapa acercaba el proyecto a su final. A medida que nuevas funcionalidades iban tomando forma y la aplicación se consolidaba, el esfuerzo invertido se transformaba en motivación para continuar con el desarrollo.

La finalización de este Trabajo Fin de Grado supone para mí mucho más que la entrega de un proyecto académico. Representa el cierre de una etapa importante y la demostración de todo lo aprendido durante la carrera. Mirando el resultado final, me siento orgulloso del trabajo realizado y de haber sido capaz de transformar una idea inicial en una aplicación completa, funcional y desplegada, que integra muchas de las tecnologías y conceptos estudiados durante estos años.

5.2 Trabajos futuros

Aunque el resultado final cumple los objetivos planteados y ofrece una experiencia satisfactoria, todavía es posible plantear mejoras. En el ámbito de la colaboración, se podría evolucionar el aspecto de compartir notas hacia un modelo de edición colaborativa en tiempo real, similar al ofrecido por herramientas como Google Docs, permitiendo que varios usuarios trabajen simultáneamente sobre un mismo documento. Por otro lado, el módulo de calendario podría mejorarse mediante la integración con servicios externos como Google Calendar o Outlook, facilitando la sincronización de eventos y tareas. Finalmente, las funcionalidades basadas en

inteligencia artificial podrían ampliarse mediante la incorporación de un asistente capaz de crear tareas o eventos a partir de instrucciones en lenguaje natural, mejorando la productividad y la experiencia de uso de la aplicación.

Bibliografía

- [1] Google, “Notebooklm,” 2025. [Online]. Available: <https://notebooklm.google.com/>
- [2] Doist Inc., “Todoist,” 2025. [Online]. Available: <https://todoist.com/>
- [3] Notion Labs Inc., “Notion,” 2025. [Online]. Available: <https://www.notion.so/>
- [4] Google, “Angular framework.” [Online]. Available: <https://angular.dev/>
- [5] VMware, “Spring boot.” [Online]. Available: <https://spring.io/projects/spring-boot>
- [6] Oracle, “Mysql.” [Online]. Available: <https://www.mysql.com/>
- [7] OpenAPI Initiative, “Openapi specification.” [Online]. Available: <https://www.openapis.org/>
- [8] OpenAI, “Gpt-4o api.” [Online]. Available: <https://platform.openai.com/>
- [9] GitHub, “Github platform.” [Online]. Available: <https://github.com/>
- [10] —, “Github actions.” [Online]. Available: <https://github.com/features/actions>
- [11] SonarSource, “Sonarqube.” [Online]. Available: <https://www.sonarsource.com/products/sonarqube/>
- [12] Docker Inc., “Docker.” [Online]. Available: <https://www.docker.com/>
- [13] Oracle, “Java se 21.” [Online]. Available: <https://www.oracle.com/java/>
- [14] Apache Software Foundation, “Apache maven.” [Online]. Available: <https://maven.apache.org/>
- [15] Swimlane, “ngx-charts.” [Online]. Available: <https://swimlane.github.io/ngx-charts/>
- [16] Mockito, “Mockito framework.” [Online]. Available: <https://site.mockito.org>
- [17] REST Assured, “Rest assured.” [Online]. Available: <https://rest-assured.io/>
- [18] Karma, “Karma test runner.” [Online]. Available: <https://karma-runner.github.io/latest/index.html>
- [19] Jasmine, “Jasmine framework.” [Online]. Available: <https://jasmine.github.io>
- [20] Selenium, “Selenium webdriver.” [Online]. Available: <https://www.selenium.dev>
- [21] Microsoft, “Visual studio code.” [Online]. Available: <https://code.visualstudio.com>
- [22] Postman, “Postman api platform.” [Online]. Available: <https://www.postman.com>
- [23] SonarSource, “Sonarcloud.” [Online]. Available: <https://sonarcloud.io>
- [24] Eclemma, “Jacoco java code coverage library.” [Online]. Available: <https://www.jacoco.org>
- [25] Kent Beck et al., “Manifesto for agile software development.” [Online]. Available: <https://agilemanifesto.org/>
- [26] Kent Beck, “Extreme programming (xp).” [Online]. Available: <https://www.extremeprogramming.org/>

Anexos



Funcionalidades detalladas

El presente anexo recoge la descripción detallada de las funcionalidades implementadas en la aplicación WrapItUp Planner. Con el objetivo de facilitar su comprensión y organización, las funcionalidades se agrupan por módulos funcionales según el área del sistema a la que pertenecen.

A.1 Gestión académica y planificación personal

Este módulo agrupa las funcionalidades relacionadas con la organización académica y la planificación personal del usuario dentro de la plataforma. Incluye herramientas destinadas a la gestión de eventos, tareas y visualización de actividad.

A.1.1 Calendario interactivo

La aplicación incorpora un calendario interactivo que permite a los usuarios registrados organizar sus actividades académicas y personales. El sistema admite la creación de eventos de un único día o de varios días consecutivos, además de la gestión de tareas diarias en formato *to-do list*. Cada evento puede ser identificado mediante un color personalizado para facilitar su visualización.

El calendario ofrece distintas vistas de navegación, permitiendo una planificación flexible y detallada del tiempo del usuario.

Las funcionalidades correspondientes a la interacción con el calendario serian las siguientes:

- Creación, modificación y eliminación de eventos.
- Gestión de eventos de un día o de varios días.
- Asignación de colores a los eventos para su identificación.
- Integración con tareas diarias tipo *to-do list*.
- Visualización mensual del calendario.
- Visualización detallada diaria mediante vista específica.

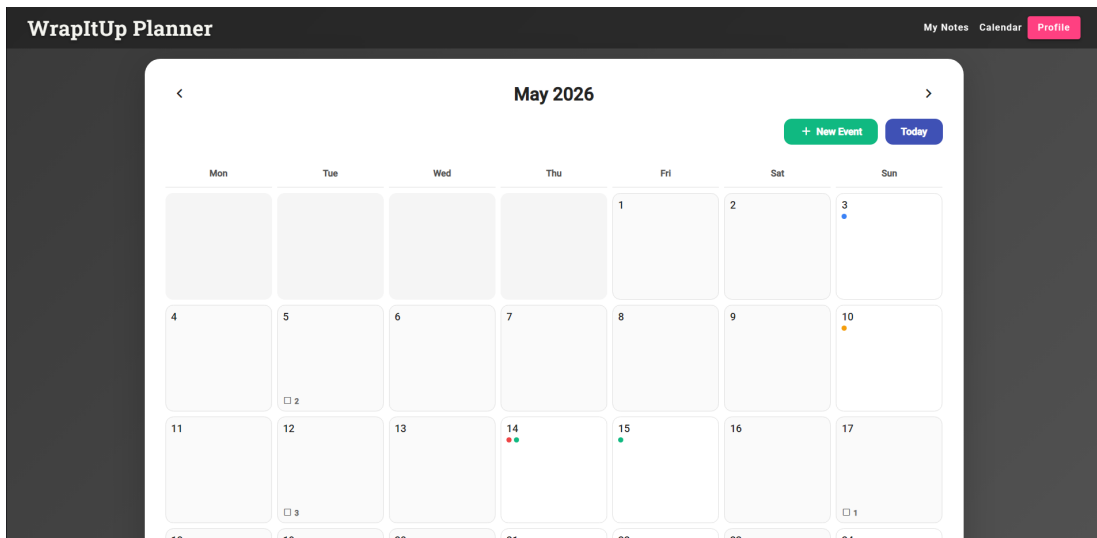


Figura A.1: Vista mensual del calendario interactivo

A.1.2 Sistema de tareas diarias

El sistema incluye una funcionalidad de gestión de tareas diarias integrada dentro de la vista diaria del calendario. Esta herramienta permite registrar, organizar y seguir actividades pendientes del usuario, facilitando la planificación del trabajo diario.

Las tareas pueden añadirse, modificarse y eliminarse dinámicamente, además de poder marcarse como completadas una vez finalizadas.

- Creación, edición y eliminación de tareas.
- Marcar tareas como completadas.
- Integración con la vista diaria del calendario.

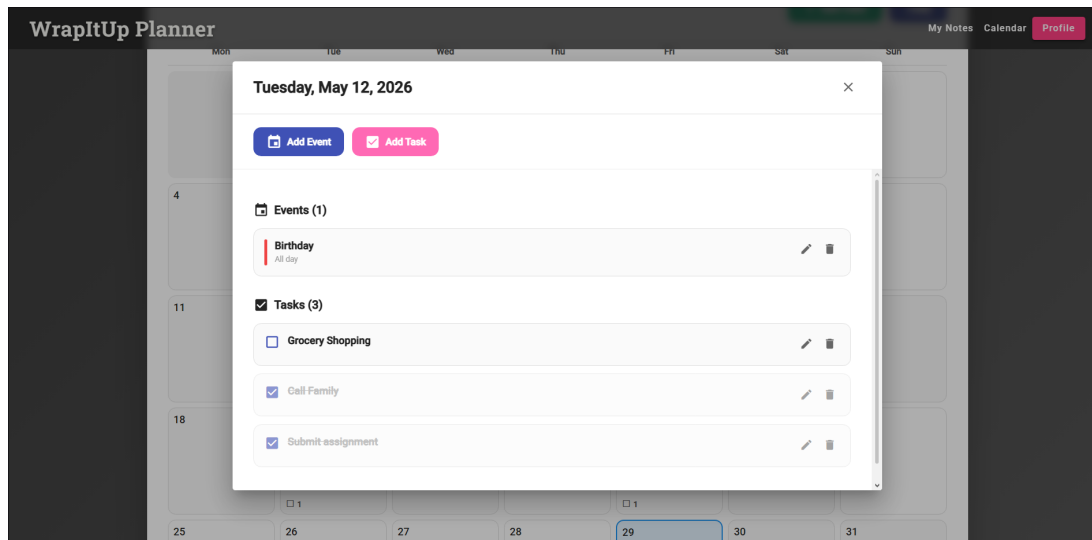


Figura A.2: Vista diaria del calendario con sistema de tareas

A.1.3 Mapa de calor de actividad

El perfil del usuario incorpora un mapa de calor (*heatmap*) que representa visualmente la actividad diaria del usuario en función de las tareas pendientes a lo largo del mes actual.

Esta visualización permite identificar patrones de productividad y obtener una visión global de la carga de trabajo del usuario.

- Representación visual de la actividad diaria.
- Basado en el número de tareas sin completar por día.
- Diferenciación de niveles de actividad.

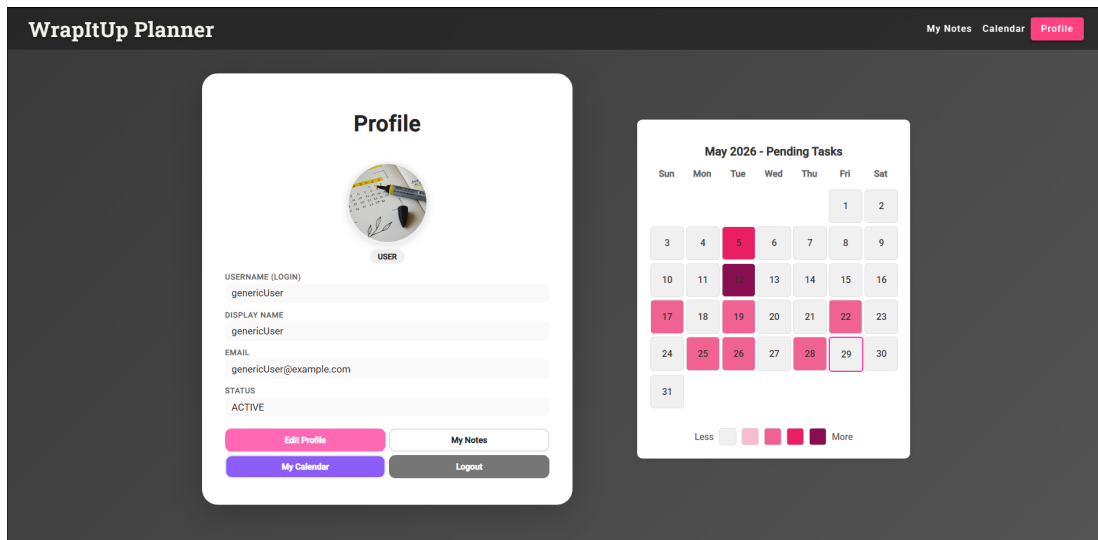


Figura A.3: Mapa de calor integrado en el perfil de usuario

A.2 Gestión de notas y contenidos

Este módulo engloba las funcionalidades relacionadas con la creación, organización, almacenamiento y procesamiento de notas dentro de la plataforma. Asimismo, incorpora las herramientas basadas en inteligencia artificial orientadas a la generación automática de contenido educativo.

A.2.1 Gestión de notas

Los usuarios registrados pueden crear, editar y eliminar notas personales dentro de la plataforma. Cada nota puede organizarse mediante categorías, facilitando así la clasificación y el acceso al contenido almacenado.

Las funcionalidades son las siguientes:

- Creación de nuevas notas.
- Edición y eliminación de contenido.
- Filtrado de notas mediante categorías.
- Visualización de notas propias.
- Compartir notas con otros usuarios.
- Acceso a notas compartidas por otros usuarios con permisos adecuados.

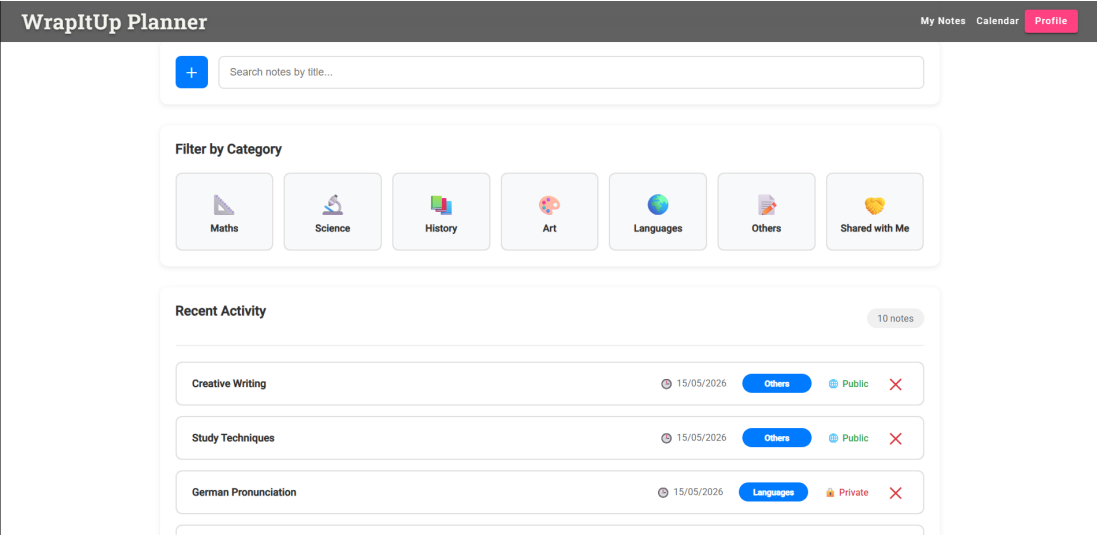


Figura A.4: Dashboard de gestión de notas

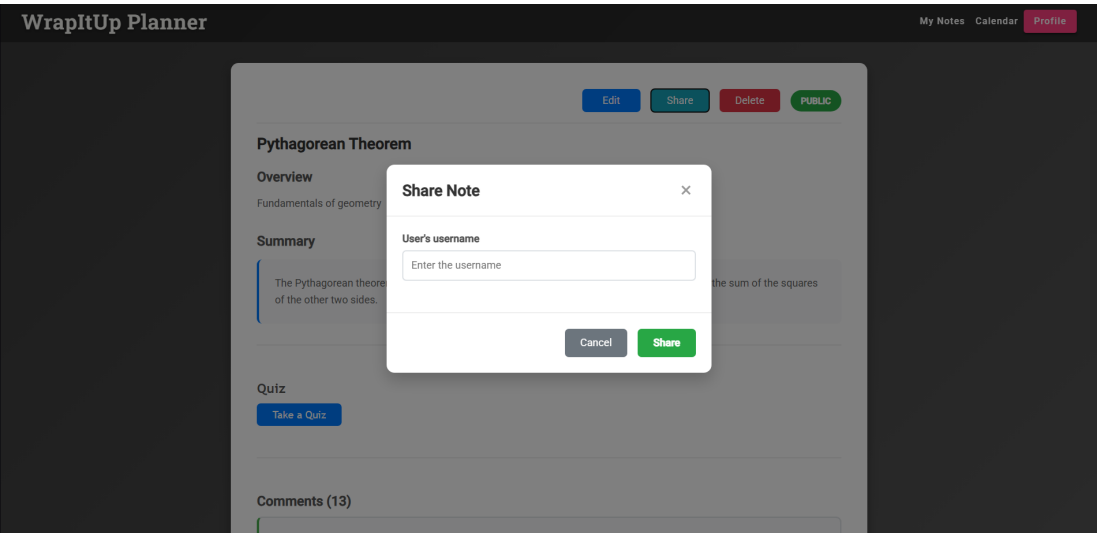


Figura A.5: Funcionalidad de compartición de notas

A.2.2 Subida y procesamiento de documentos

La plataforma permite subir documentos para su posterior procesamiento automático mediante modelos de inteligencia artificial. A partir del contenido proporcionado por el usuario, el sistema genera distintos recursos de apoyo al estudio.

El procesamiento automático incluye:

- Generación de una descripción general del contenido.
- Creación de un resumen estructurado.
- Generación de preguntas tipo test.

Cabe destacar que la generación de preguntas tipo test no se limita únicamente a las notas generadas mediante inteligencia artificial, sino que también puede realizarse en notas creadas manualmente a partir de material fuente proporcionado directamente por el usuario.

Esta funcionalidad permite transformar notas tradicionales en recursos interactivos orientados al aprendizaje y la autoevaluación.

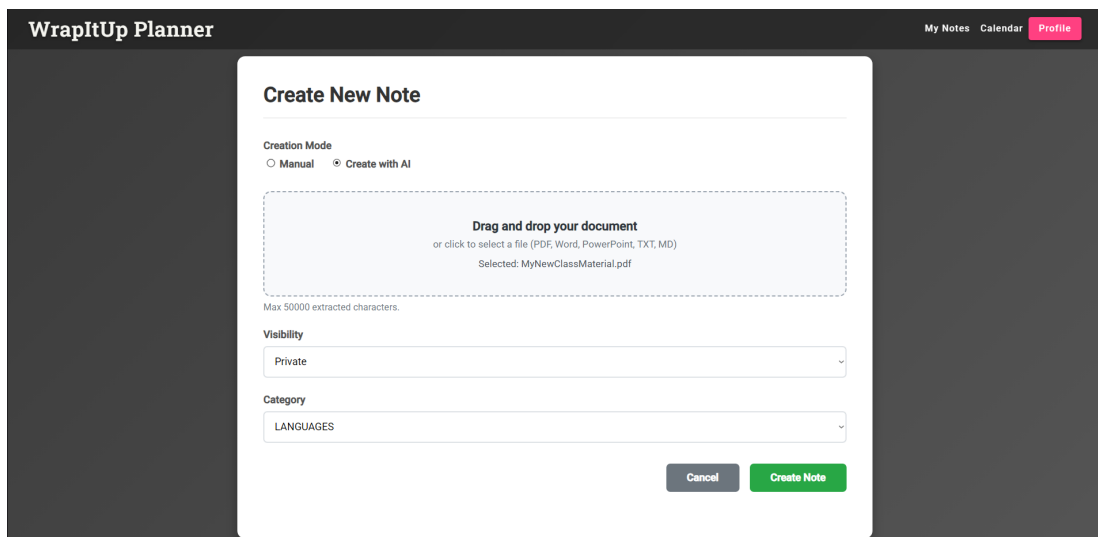


Figura A.6: Pantalla de subida y procesamiento de documentos

A.3 Aprendizaje y seguimiento del progreso

Este módulo reúne las funcionalidades orientadas al refuerzo del aprendizaje y al seguimiento del progreso académico del usuario mediante herramientas interactivas basadas en inteligencia artificial.

A.3.1 Cuestionarios interactivos

A partir de los documentos procesados por el sistema, la aplicación genera automáticamente cuestionarios tipo test compuestos por preguntas de opción múltiple. Estos cuestionarios permiten realizar sesiones de autoevaluación directamente desde la plataforma utilizando contenido derivado del material educativo del usuario.

Las funcionalidades principales incluyen:

- Generación automática de preguntas mediante IA.
- Realización de cuestionarios interactivos.
- Corrección automática de respuestas.
- Visualización inmediata de resultados.

Tanto usuarios registrados como usuarios no registrados con acceso a una nota compartida pueden completar los cuestionarios asociados al contenido.

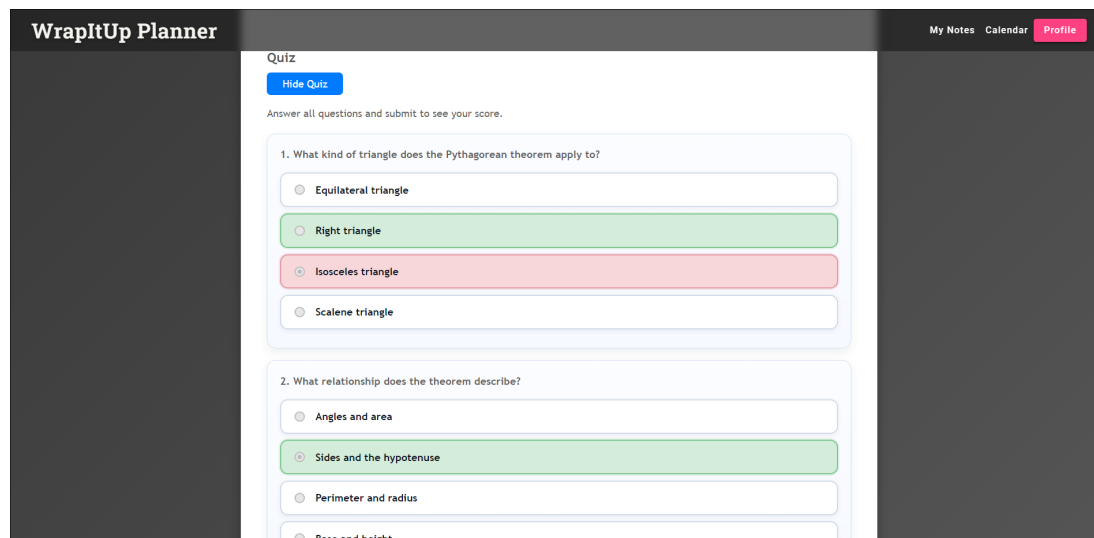


Figura A.7: Cuestionario interactivo generado mediante inteligencia artificial

A.3.2 Seguimiento del progreso

La plataforma registra los resultados obtenidos por los usuarios registrados en los distintos cuestionarios realizados, permitiendo representar visualmente la evolución de su rendimiento a lo largo del tiempo.

Para ello, se incorpora un gráfico lineal que muestra la progresión de las puntuaciones obtenidas en diferentes sesiones de estudio, facilitando así el seguimiento del aprendizaje y la identificación de mejoras en el rendimiento académico.

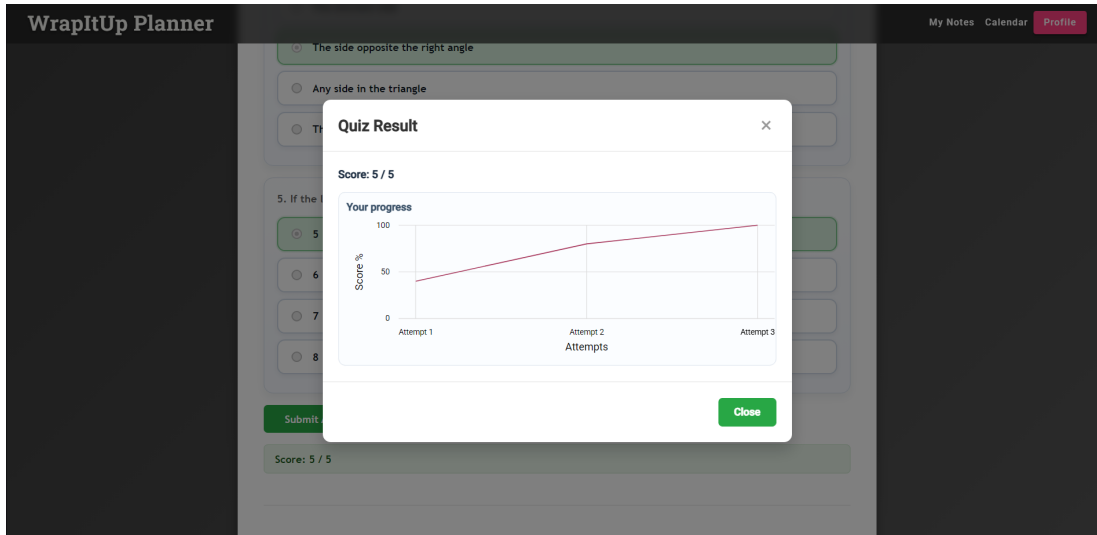


Figura A.8: Gráfico de evolución del rendimiento en cuestionarios

A.4 Interacción social y colaboración

Este módulo agrupa las funcionalidades orientadas a la comunicación entre usuarios dentro de la plataforma, así como los mecanismos de control y moderación del contenido generado en la interacción social.

A.4.1 Sistema de comentarios

Los usuarios registrados pueden interactuar mediante un sistema de comentarios asociado a las notas compartidas. Esta funcionalidad permite la discusión y el intercambio de información entre usuarios que tengan acceso al contenido.

Las principales características del sistema son:

- Publicación de comentarios en notas compartidas.
- Visualización de comentarios por parte de usuarios con acceso a la nota.
- Eliminación de comentarios por parte del autor o administradores.

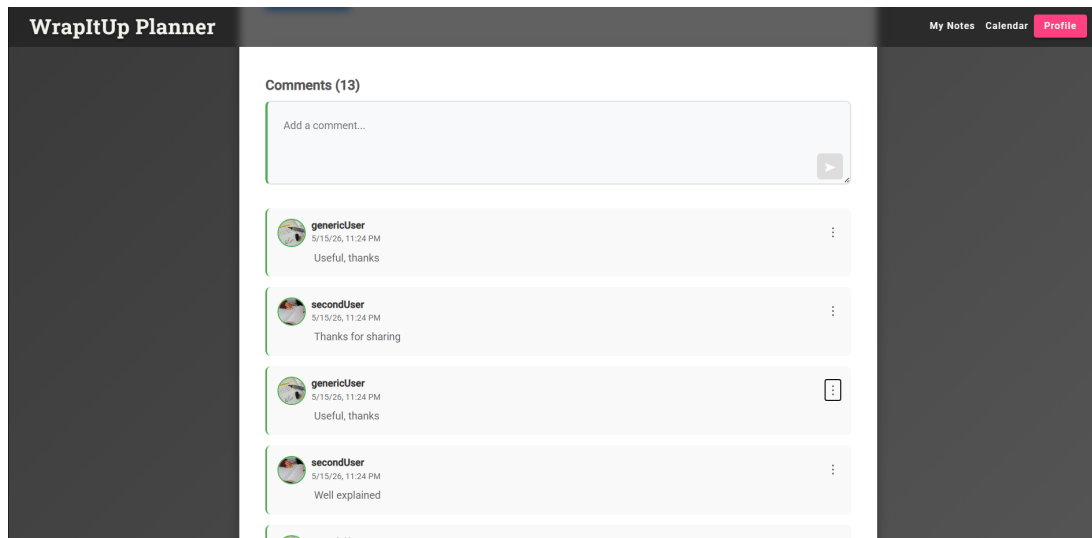


Figura A.9: Sistema de comentarios en una nota

A.4.2 Sistema de reportes

Con el objetivo de garantizar un entorno seguro dentro de la plataforma, los usuarios pueden reportar comentarios que consideren inapropiados. Estos reportes son almacenados y gestionados posteriormente por el sistema de administración.

A.5 Administración y moderación

Este módulo engloba las funcionalidades destinadas a la gestión global de la plataforma, la supervisión del contenido y la aplicación de medidas de moderación para garantizar el correcto uso del sistema.

A.5.1 Panel de administración

Los usuarios con rol de administrador disponen de un panel de control desde el cual pueden gestionar los distintos elementos de la plataforma, incluyendo usuarios, notas y reportes generados por el sistema de interacción social.

Las funcionalidades del panel son:

- Visualización y gestión de reportes de comentarios.
- Acceso a los perfiles de usuario y a las notas asociadas a los comentarios reportados.

- Gestión de incidencias mediante la revisión y descarte de reportes considerados falsos positivos.
- Eliminación de comentarios en caso de incumplimiento de las normas de la plataforma.

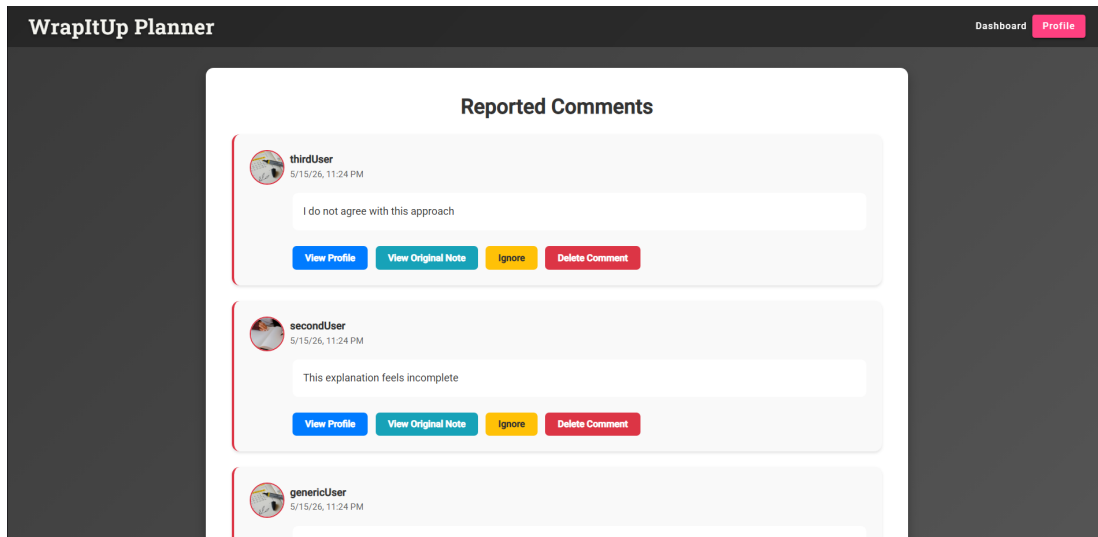


Figura A.10: Panel de administración del sistema

A.5.2 Gestión de usuarios

Los administradores cuentan con permisos avanzados que les permiten gestionar las cuentas de usuario dentro de la plataforma, asegurando el cumplimiento de las normas de uso establecidas.

Las acciones posibles son las siguientes:

- Bloqueo y desbloqueo de usuarios en caso de comportamiento inadecuado.
- Control global del acceso a las notas y perfiles de todos los usuarios.

Cuando un usuario es bloqueado, su acceso a la plataforma queda restringido automáticamente, mostrando una pantalla informativa que indica su estado de suspensión.

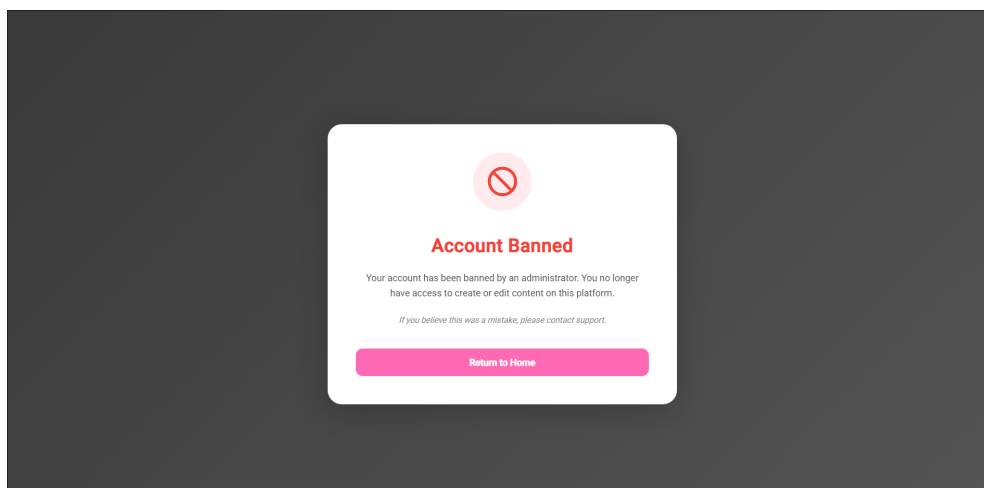


Figura A.11: Pantalla de usuario bloqueado

B

Instrucciones para ejecutar la aplicación

El presente anexo recoge las instrucciones necesarias para la instalación, configuración y ejecución de la aplicación en un entorno local.

B.1 Requisitos previos

Para poder ejecutar la aplicación en un entorno local, es necesario disponer previamente de Docker instalado en el sistema. Docker se emplea como tecnología base para la creación y gestión de contenedores, lo que permite encapsular tanto la aplicación como sus dependencias y garantizar un entorno de ejecución consistente independientemente del sistema operativo.

En función del sistema operativo utilizado, la instalación puede realizarse de la siguiente manera:

- **Windows y macOS:** se recomienda la instalación de Docker Desktop, que proporciona una interfaz gráfica y todas las herramientas necesarias para la gestión de contenedores (<https://www.docker.com/products/docker-desktop/>).
- **Linux:** es necesaria la instalación de Docker Engine junto con Docker Compose, que permiten la creación y orquestación de contenedores desde línea de comandos (<https://docs.docker.com/engine/install/>, <https://docs.docker.com/compose/install/>).

B.2 Ejecución de la aplicación

El despliegue de la aplicación en un entorno local se realiza mediante Docker, lo que permite automatizar la descarga de las imágenes necesarias y la creación de los contenedores correspondientes. Este proceso garantiza que la aplicación se ejecute en un entorno aislado y reproducible, independientemente de la configuración del sistema anfitrión.

A continuación, se detallan los comandos necesarios para llevar a cabo el despliegue:

```
1 docker pull arturox2500/wrapitup_planner:latest
2 docker pull arturox2500/wrapitup_planner-compose:latest
3
4 docker create --name temp-compose arturox2500/wrapitup_planner-compose:latest
   cmd.exe
5 docker cp temp-compose:/docker-compose.yml ./docker-compose.yml
6 docker rm temp-compose
7
8 docker compose up -d
```

Código B.1: Comandos de despliegue de la aplicación

B.3 Configuración del entorno

La aplicación requiere la definición de determinadas variables de entorno para su correcto funcionamiento. Estas variables se almacenan en un fichero `.env`, ubicado en el mismo directorio que el archivo `docker-compose.yml`, y permiten parametrizar tanto la configuración de la base de datos como distintos servicios utilizados por la aplicación.

A continuación, se agrupan las variables más relevantes:

```
1 # Base de datos
2 MYSQL_ROOT_PASSWORD=<value>
3 MYSQL_DATABASE=<value>
4 MYSQL_USER=<value>
5 MYSQL_PASSWORD=<value>
6
7 # Spring Boot
8 SPRING_DATASOURCE_URL=<value>
9 SPRING_DATASOURCE_USERNAME=<value>
10 SPRING_DATASOURCE_PASSWORD=<value>
11 SPRING_JPA_HIBERNATE_DDL_AUTO=<value>
12 SERVER_PORT=<value>
13 SERVER_SSL_KEY_STORE_PASSWORD=<value>
14
15 # Servicios externos
16 OPENAI_API_KEY=<value>
```

Código B.2: Variables de entorno principales

B.4 Acceso a la aplicación

Una vez que los contenedores han sido correctamente desplegados y se encuentran en ejecución, la aplicación queda accesible a través del navegador web. En este punto, todos los servicios necesarios ya están inicializados y la aplicación está lista para su uso.

Aplicación web: `https://localhost:443`

B.5 Credenciales de ejemplo

La aplicación incorpora un conjunto de usuarios de prueba con el objetivo de facilitar la evaluación de las distintas funcionalidades implementadas. Estos usuarios permiten probar los diferentes roles disponibles en el sistema sin necesidad de realizar un proceso de registro previo.

Nombre de usuario	Contraseña	Rol
genericUser	12345678	Usuario
secondUser	12345678	Usuario
thirdUser	12345678	Usuario
admin	12345678	Administrador

Tabla B.1: Credenciales de acceso de los usuarios de prueba

B.6 Datos de ejemplo

La aplicación se inicializa con datos de ejemplo para demostrar su funcionalidad:

Usuarios:

- Usuario principal con varias notas, eventos y tareas.
- Usuarios secundarios con menor volumen de contenido.
- Administrador con acceso completo al sistema.

Notas:

- Conjunto de notas académicas en distintas categorías.
- Mezcla de contenido público y privado.

- Notas compartidas entre usuarios para colaboración.

Comentarios:

- Interacciones entre usuarios en distintas notas.
- Algunos comentarios reportados para moderación.

Calendario:

- Eventos académicos y personales.
- Eventos de uno o varios días.
- Actividades de estudio y organización.

Tareas:

- Tareas diarias de planificación académica.
- Mezcla de tareas completadas y pendientes.
- Visualización mediante mapa de calor en el perfil.



Ejecución y edición de código

El presente anexo describe el procedimiento necesario para configurar el entorno de desarrollo, ejecutar la aplicación y lanzar las pruebas a partir del código fuente del proyecto.

C.1 Clonado del repositorio

En primer lugar, se procede a la clonación del repositorio del proyecto y al acceso al directorio principal del mismo:

```
1 git clone https://github.com/codeurjc-students/2025-WrapItUp-Planner.git
2 cd 2025-WrapItUp-Planner
```

Código C.1: Clonado del repositorio

C.2 Ejecución del sistema

C.2.1 Base de datos y servicios

La aplicación utiliza **MySQL** como sistema gestor de bases de datos. Los datos de ejemplo se inicializan automáticamente mediante la clase `DatabaseInitializer.java` durante el arranque de la aplicación.

Las credenciales de conexión configuradas por defecto son las siguientes:

- **Usuario:** user
- **Contraseña:** password
- **Base de datos:** wrapitup
- **URL de conexión:** jdbc:mysql://localhost:3306/wrapitup

Estas credenciales corresponden a la configuración por defecto del entorno de desarrollo.

La base de datos puede desplegarse mediante un contenedor Docker o mediante una instalación local de MySQL.

C.2.2 Ejecución del backend (Spring Boot)

Para la ejecución del *backend* se emplean los siguientes comandos:

```
1 cd backend/wrapitup_planner
2 mvn clean install
3 mvn spring-boot:run
```

Código C.2: Ejecución del backend

Por defecto, el servicio *backend* se encuentra disponible en:

https://localhost:443

C.2.3 Ejecución del frontend (Angular)

La ejecución del *frontend* requiere la instalación de dependencias y el arranque del servidor de desarrollo:

```
1 cd frontend/WrapItUp-Planner
2 npm install
3 ng serve
```

Código C.3: Ejecución del frontend

La aplicación web queda disponible en:

http://localhost:4200

C.2.4 Herramientas de desarrollo

Se recomienda el uso de **Visual Studio Code** como entorno de desarrollo, debido a su amplia compatibilidad con Angular y Spring Boot. Para la interacción con la API REST se utiliza **Postman**, cuya colección se encuentra disponible en el directorio de documentación del proyecto.

C.2.5 Ejecución de pruebas

Pruebas del backend

En el backend se han definido distintos niveles de pruebas para validar la lógica de negocio, la integración con la base de datos y el funcionamiento de la API REST.

- **Pruebas unitarias** (*unit*): verifican los servicios de forma aislada mediante *mocks*, implementadas con **JUnit 5** y **Mockito**.
- **Pruebas de integración** (*integration*): validan la API REST y la interacción entre componentes levantando el contexto completo de Spring, utilizando **RestAssured**.
- **Pruebas de sistema** (*system*): comprueban flujos completos del *backend* en un entorno controlado, usando **SpringBootTest** y transacciones para el aislamiento de datos.
- **Pruebas end-to-end web** (*e2e*): validan la interacción con el *frontend* mediante la simulación de flujos de usuario con **Selenium WebDriver**.

Pruebas del frontend

En el *frontend* se han realizado pruebas para validar los componentes y su integración con la API REST.

- **Pruebas de componentes**: verifican la renderización y comportamiento de los componentes de forma aislada mediante *mocks* de servicios, usando **Karma** y **Jasmine**.
- **Pruebas de servicios**: validan la comunicación con la API REST, comprobando la correcta gestión de peticiones y respuesta de datos.

Ejecución de pruebas

Para la ejecución de las pruebas se utilizan los siguientes comandos:

```
1 cd frontend/WrapItUp-Planner
2 npm install
3 npm run test:ci
```

Código C.4: Pruebas del frontend

Este *script* ejecuta todas las pruebas del *frontend* en modo continuo y sin interfaz gráfica.

```
1 cd backend/wrapitup_planner
2 mvn clean test
```

Código C.5: Ejecución de pruebas del backend

Este comando corresponde a la ejecución de las pruebas del *backend*.